Highlights

SolidSessionBench: Realistic Query Sequences for User-Oriented Decentralized Environments (Extended)

Ruben Eschauzier, Ruben Taelman

- Introduces SolidSessionBench, a benchmark simulating realistic query sequences to evaluate client-side optimization in decentralized environments.
- Models empirical user interactions using logical query sessions, interest-driven data locality, and dynamic query refinements.
- Implements baseline client-side caching strategies and cache-based cardinality estimation for Link Traversal Query Processing (LTQP).
- Demonstrates that realistic sequence simulation reveals critical performance dynamics that traditional, non-sequence-based workloads miss.

SolidSessionBench: Realistic Query Sequences for User-Oriented Decentralized Environments (Extended)

Ruben Eschauzier[a,*], Ruben Taelman[a]

[a]Department of Electronics and Information Systems, Ghent University – imec, Universiteitstraat 4, Ghent, 9000, Flanders, Belgium

ARTICLE INFO

Keywords:
Multi-perspective data
Link Traversal-based Query Processing
Client-side caching
Query sequence simulation

ABSTRACT

Emerging decentralized architectures give users complete control over their own data, spreading information across heterogeneous sources. Without a central authority to enforce consistency, users freely publish overlapping and potentially conflicting claims, generating a multi-perspective decentralized environment. Integrating this fragmented data is challenging because no single authoritative endpoint exists to aggregate results or resolve conflicts. Client-side query engines overcome this by shifting execution locally, enabling users to dynamically fetch and reconcile information on the fly. However, optimizing this execution remains challenging due to network latency and a lack of prior knowledge regarding data distribution. Client-side caching mitigates these issues by exploiting data access patterns across query sequences, but accurately evaluating these strategies is difficult. Existing evaluations rely on static micro-benchmarks or random sequences that fail to capture the behavioral variability of real users. To address this, we evaluate the performance of four client-side caching strategies and two cache-based cardinality estimation techniques for Link Traversal Query Processing (LTQP). We conduct this evaluation using SolidSessionBench, an evidence-based benchmark that simulates realistic user query sequences to test caching algorithms under empirical usage patterns. Our evaluation reveals critical performance dynamics: session switching severely degrades cache hit rates, and deep exploratory query refinement introduces significant cache invalidation challenges. Despite these challenges, caching remains a highly effective optimization approach that significantly improves performance over the baseline. Furthermore, we demonstrate that traditional workloads produce misleading, overly pessimistic performance estimates. We conclude that optimizing LTQP requires session-aware caching architectures, and that modeling realistic user interactions is essential for accurately evaluating these techniques.

1. Introduction

Emerging web applications increasingly rely on decentralized architectures to give users control over their personal data. Structured decentralized ecosystems, such as Solid [49] and Mastodon [46], encourage users to keep their data in small, independent stores following shared data models. This paradigm shift is increasingly vital for automated systems, such as LLM-based AI agents, which query these decentralized structures to extract facts for task performance or question answering [61, 34, 10]. While this approach strengthens user autonomy, the absence of a central data authority naturally creates a highly multi-perspective environment. Data spreads across thousands of heterogeneous sources that frequently contain overlapping, diverse, or potentially conflicting claims. Query engines must therefore dynamically fetch and integrate this information at runtime [54], not only to discover conflicting statements and multiple perspectives, but to reconcile these claims to provide a coherent answer to the user or agent.

Traditional federated querying approaches [52, 50, 32] support the integration of decentralized data sources, but they typically assume a small (10-100) set of well-described and consistently available endpoints [17]. These assumptions fail once data is spread across thousands of small units that enforce local access-control policies. In addition, enforcing fine-grained access control policies [21] on large sources with expressive interfaces introduces notable overhead [43], making standard approaches unsuitable for highly decentralized environments.

Newer decentralized environments, such as Solid, often expose data as lightweight HTTP resources instead of highly expressive query interfaces. Many of these resources are not known in advance and can only be discovered during query execution. While their simplicity lowers the cost of enforcing fine-grained controls, this setting requires a federation model that can operate over a very large number of small sources, many of which may be encountered at runtime, without relying on prior aggregation. Traditional federated approaches are insufficient in this setting due to the assumption of a small number of large endpoints. Additionally, these approaches often assume prior knowledge of the sources, which is not feasible in decentralized environments where discovery is a key requirement. Executing SPARQL queries using Link Traversal-based Query Processing (LTQP) [30] fits these needs. It discovers documents incrementally by following URIs encountered during execution, starting from an initial set of seed documents provided by either the user or the query. Its link-driven, asynchronous traversal supports fine-grained permissions and allows systems to discover without knowing the location of all decentralized data sources beforehand.

ruben.eschauzier@gmail.com (R. Eschauzier); ruben.taelman@ugent.be (R. Taelman)
www.rubensworks.net (R. Taelman)
ORCID(s): 0000-0001-0000-0000 (R. Eschauzier); 0000-0001-5118-256X (R. Taelman)

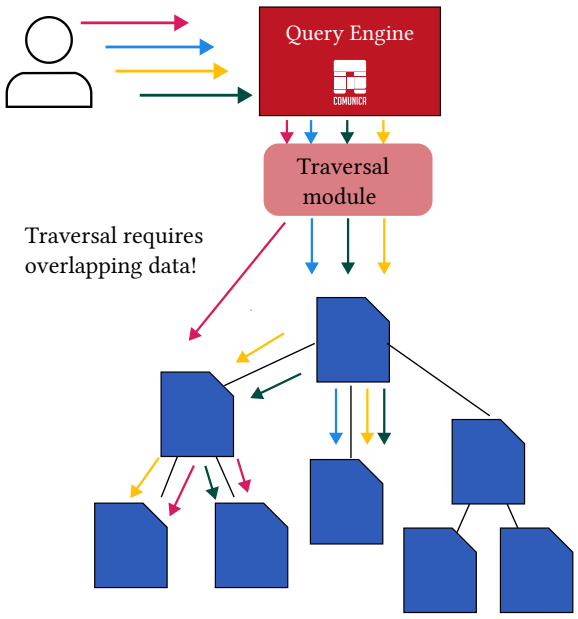

Figure 1: When users interact with query engines, their queries intentions and thus relevant data to the query can overlap, leading to opportunities for personalized optimization.

However, the fact that LTQP discovers documents incrementally creates significant challenges for efficient execution. Using the traditional optimize-then-execute query processing paradigm is difficult, as the query itself determines which data will be accessed. As a result, the optimizer has no prior knowledge of the relevant data until the query's traversal has already begun. Some approaches optimize query execution without prior knowledge by using zero-knowledge query planning [28] or adaptive query processing [26]. These yield modest but inconsistent performance gains, depending on the decentralized environment and the heuristics used [57]. However, other aspects of zero-knowledge LTQP optimization, such as link prioritization [31], are unlikely to improve query result arrival times in emerging decentralized environments [20].

Consequently, other works explore using precomputed server-side information as prior knowledge or relevance indexes to efficiently execute LTQP queries [58, 59, 25]. While these approaches offer greater performance benefits, they require the server to maintain these indexes. This raises the cost of serving resources and complicates privacy and access control.

To avoid server-side overhead, engines can derive prior knowledge directly on the client. In decentralized settings, LTQP operates inherently client-side, with a single engine instance serving one or more applications, often specific to a particular user. As these users and thus the used applications issue sequences of correlated queries, the engine identifies frequently accessed data and its characteristics. This allows engines to implement *personalized* query optimization algorithms that leverage local access patterns to build necessary prior knowledge.

Client-side caching is well-known mechanism to exploit these access patterns and mitigate network latency. However, accurately evaluating caching strategies requires realistic query sequences. Traditional benchmarks execute isolated queries or random sequences, which fail to capture client-side dynamics accurately. Consequently, these methods yield misleading estimates of cache performance in real-world scenarios.

In this paper, we present a comprehensive evaluation of client-side caching strategies for LTQP. We introduce SolidSessionBench to generate realistic query sequences by modeling empirical usage patterns, including logical query sessions, interest-driven data locality, and dynamic query refinements. Conducting our experiments using these sequences ensures our evaluation reflects real-world behavior. By evaluating caching against these sequences, we uncover important performance dynamics that traditional benchmarks miss.

In summary, this paper makes the following contributions:

- We implement four client-side caching strategies (three in-memory and one disk-based) and two cache-based cardinality estimation techniques for Link Traversal Query Processing.
- We evaluate these approaches, demonstrating that client-side caching significantly improves query execution times and throughput despite a lack of prior knowledge.
- We highlight how realistic sequence simulation fundamentally alters performance evaluations, proving that traditional, non-sequence-based workloads yield overly pessimistic estimates.
- We introduce SolidSessionBench, an evidence-based query sequence simulator, to validate these caching strategies against realistic user interactions.

This article extends our previous work presented at the International Semantic Web Conference (ISWC) 2026 [19]. While the initial study introduced the SolidSessionBench methodology and evaluated standard in-memory caches, this version shifts the focus to a comprehensive investigation of client-side caching architectures. Specifically, we introduce in-memory and disk-based quad store caches to unify the indexes of the cached triples. Furthermore, we improve the original cache-based cardinality estimation technique by incorporating cache entry reachability to generate more accurate estimates. Finally, we expand the experimental analysis by applying a linear mixed-effects regression model to quantify the exact performance impact of individual query execution chokepoints on the cache.

The remainder of this paper is structured as follows. Section 2 reviews prior benchmarking approaches and real-world query usage patterns. Section 3 outlines our simulation methodology and defines the resulting hyperparameters. Section 4 introduces the technical mechanics of client-side

caching strategies in LTQP. Section 5 evaluates the benchmark using proxy measures of realism. Finally, Section 6 concludes the paper and presents a plan for benchmark sustainability.

2. Literature Review

To ground our benchmark design in existing research and ensure a realistic sequence generator, this section reviews prior benchmarking approaches for client-side query optimization and real-world query usage patterns.

2.1. Existing Benchmarking Approaches

For the evaluation of client-side benchmarking techniques that rely on previous queries, we primarily investigate caching literature, as this is a well-studied problem relevant to our work.

In the literature, we find three distinct benchmarking approaches: simulated micro benchmarks, simulated end-to-end query benchmarks, and real-world query logs.

Simulated micro benchmarks attempt to isolate performance gains of the benchmarked approaches by fixing certain aspects of query execution. This can range from fixing the cached data size [27] to controlling the percentage of query-relevant data present in the cache [41]. Fixing aspects of query execution makes it easier to isolate and understand the effect of specific caching mechanisms. However, it also gives an incomplete picture, because we do not know how those fixed conditions relate to real workloads or how the methods perform when those assumptions do not hold.

Simulated end-to-end query benchmarks use generated query sequences to evaluate performance. They aim to measure cache hit rate and overall execution behavior under the assumption that the simulated sequences reflect realistic usage. In practice, the simulation is usually built by extending an existing benchmark so that it generates sequences instead of isolated queries. The sequence order is often determined by simple rules or by random selection.

A common approach is to use queries from existing benchmarks such as the Star Schema Benchmark [42] or the Berlin SPARQL Benchmark [11]. These queries are then randomly divided into sequences [27, 41, 23], with each sequence executed using a shared cache. This approach is straightforward but not realistic. Prior work shows that real workloads contain user behavior patterns and temporal locality, and these patterns can be exploited by caching methods. Random sequences fail to capture these properties.

More realistic is to adjust the query generators of existing benchmarks to include patterns in the query sequences. Some works [62, 37] use a Pareto distribution to decide what entity is used to instantiate a query template. This follows from the observation that many human activities follow a Pareto distribution [6], with many queries relating to a small portion of the entities.

A third approach uses simulated query sequences that reflect the access patterns of specific applications. One example is a workload that models data scientists exploring different aspects of the same entity, such as an author in a scholarly network, through consecutive, thematically related queries [15]. Such simulations often include multiple query sessions, with new sessions beginning according to a probability parameter p. Another example is a simulated application scenario focused on retrieving information about vacation destinations, which is used to assess caching effectiveness [37].

Real-world query logs Whenever available, query logs from actual applications form a highly relevant benchmarking approach. However, obtaining these query logs can be challenging. For SPARQL query optimization, query logs for a variety of endpoints are available, such as DBpedia [48, 53, 9] and Wikidata [53]. These logs are extensively used in benchmarking SPARQL-based approaches [63, 44, 35].

The evaluation of personalized LTQP optimization would ideally use real-world query logs, but such logs do not exist. To avoid the fragmented evaluation practices of current simulated approaches, the next best option is a unified simulated benchmark. A shared benchmark would provide a common basis for comparison, improve reproducibility, and prevent each paper from recreating its own evaluation setup.

2.2. Real-world Query Usage Patterns

To inform the benchmark design on what patterns should be simulated, we will outline three relevant client query usage patterns observed in the literature. The relevance is determined for our social network application use case.

User Interest-Based Query Behavior Work on social networks shows users form communities based on shared interests [39, 16, 7, 33]. These networks exhibit power-law degree distributions, small-world structures, and scale-free properties. This topology creates a dense core of high-degree nodes that connect many smaller, strongly clustered groups [39]. Simulation studies confirm that this interest-driven group formation is stable and reliably modeled [2, 24].

Furthermore, empirical research formally establishes that shared interests directly drive a higher degree of explicit and implicit interactions between users [40]. These findings justify our simulation of user-relatedness and interest-driven behavior when generating query sequences.

Logical Query Sessions Analysis of web browsing behavior shows that activity is best grouped into *logical sessions*: sequences of user interactions focused on specific goals, rather than purely time-bound intervals [38, 47, 51]. These task-oriented sessions frequently involve non-linear navigation, including backtracking. New sessions begin in approximately 15% of clicks, following a log-normal distribution with average size and depth means of 6.1 and 3, respectively.

These patterns also appear in application navigation. In a decentralized social media application, for example, a user viewing a feed might click a creator's username, view comments, or like a post. Because the parameters of each new query derive directly from the preceding query's results, these queries chain together into structures mirroring logical

web sessions. Additionally, direct searches for specific posts or topics reflect standard session-switching behavior.

Consequently, interactive applications generate sequences of interdependent requests [60]. This dynamic informs the design of Facebook's TAO data store, which is explicitly optimized for workloads driven by the user's step-by-step traversal of the social graph within the interface [14].

Query Refinement Analysis of SPARQL queries issued to the DBpedia endpoint reveals that users frequently submit related queries in streaks [13, 64]. These streaks consist of query sequences that share an underlying intention but undergo iterative refinement to achieve the user's objective. Previous studies initially introduced the concept of SPARQL query sessions specifically to analyze robotic queries [64, 13].

In this benchmark, we focus on organic query sequences [64], which users generate through interactions with software or browsers [36]. Because organic interactions are driven by application design rather than underlying data organization, we assume these user behaviors will also emerge in applications navigating structured decentralized environments Furthermore, we deprioritize robotic queries because their sequences can already be simulated by traditional benchmark approaches [57, 11, 1].

For both organic and robotic queries, analysis of total usage and reformulations (removals and additions) of operators over all query logs shows that filter, union, and optional operators account for the vast majority of reformulated queries.

Recent analyses of Wikidata query logs further demonstrate that query development is an iterative process of design, debugging, and maintenance [5]. Based on this empirical data, exploratory querying encompasses six core behaviors [5]: **result refinement** to reduce records via filters or selective triple patterns, **result generalization** to retrieve more entities by broadening parameters, such as adding superclasses or increasing limits, **result extension** to fetch additional attributes by appending new triple patterns, **bug fixing** to correct structural errors like invalid URIs, **query refinement** to adjust complex language features, and **shifting query intent** to pivot topics by altering core predicates.

These findings confirm that real-world querying relies on dynamic, step-by-step modifications rather than the static execution of independent templates. Consequently, existing evaluation frameworks fail to capture how users actively interact with data systems. Evaluating system performance for these exploratory query sequences remains a critical research challenge, explicitly requiring new benchmarks to validate progress [5]. Therefore, modeling these empirical sequence patterns is essential to accurately assess client-side query optimization techniques.

3. Query Sequence Benchmark

In this section, we detail our simulation methodology and the resulting hyperparameters. We base our sequence generator on SolidBench [57], which simulates realistic decentralized social media applications within the Solid ecosystem. SolidBench utilizes the Social Network Benchmark (SNB) dataset [18, 3] and applies the Linked Data Benchmark Council (LDBC) choke point methodology [4].

The choke point methodology is a benchmark design strategy that targets specific query execution and optimization bottlenecks in current database management systems [12]. Inspired by classical benchmarks like TPC-H, this approach uses these technical challenges to guide research and system development. For graph data, choke points include estimating cardinality during traversals with data skew, determining optimal join orders, and handling scattered index access patterns that lack predictable locality [18]. In practice, a choke point-based benchmark design strategy first identifies critical execution challenges, then designs queries or modifications that explicitly test them.

We evaluate our prioritization algorithms using SolidBench [57], the state-of-the-art benchmark for structured decentralized environments. SolidBench simulates a social network containing users, posts, and comments, all described by a single ontology. To represent Solid pods, the benchmark fragments data so that all information related to a specific user resides in a single pod. Under default settings, SolidBench generates 158,233 RDF files across 1,531 data vaults, totaling 3,556,159 triples.

To simulate a highly distributed network, SolidBench partitions the Social Network Benchmark (SNB) dataset into individual data pods. Its sequential architecture executes in four modular steps:

1. The LDBC Datagen generates the underlying dataset.
2. An enhancer creates additional data, such as noise triples that do not participate in queries.
3. A dataset fragmenter partitions the triples into a Solid-like structure.
4. The SPARQL queries are instantiated.

The workload combines standard SNB *interactive* queries (*short* and *complex*) with custom *discover* templates that explicitly target Link Traversal Query Processing (LTQP) choke points. During execution, a client-side engine dynamically traverses links between these scattered pods to resolve the queries.

To support sequence-based evaluation, we significantly extend and restructure three core SolidBench components:

- **Query Generator:** Modified to produce correlated query sequences that reflect empirical usage patterns.
- **Dataset Generator:** Enhanced to model interest-based similarities among simulated users.
- **Benchmark Runner:** Updated with sequence execution logic to facilitate reproducible experiments and simplify the generation of default configuration files.

Link traversal engines cannot currently solve *complex* queries in a feasible timeframe, so we omit them from our

default benchmark configuration. However, because the sequence generator is fully configurable, researchers can easily include complex query templates in future benchmarks as traversal engines improve.

3.1. Realism of Query Sequences

To ensure SolidSessionBench accurately reflects user behavior in decentralized environments, we base our simulation methodology on the LDBC methodology used for data generation. The sequence generator synthesizes empirical usage patterns established in Section 2 to form a grounded simulation methodology that reflects real-world patterns.

Because direct, real-world query logs for highly decentralized LTQP environments do not currently exist, we maintain and evaluate realism through a three-pillared approach:

- **Topological Realism (User Interests):** We utilize the established LDBC social network topology to model explicit and implicit user relatedness, ensuring data access patterns mirror natural community formations rather than uniform distributions. Due to the proven realism of the LDBC data generator [45], our simulations inherit this empirically proven structure.
- **Navigational Realism (Logical Sessions):** We restrict query sequences to directed dependency graphs. This simulates realistic "click-through" application navigation, accounting for empirically observed session lengths and log-normal transition probabilities.
- **Exploratory Realism (Query Refinement):** We replace static templates with dynamic query transformations. Mapping real-world SPARQL refinements (e.g., result generalization, intent shifting) directly to execution choke points ensures engines face realistic structural variability.

We evaluate the success of this simulation by measuring proxy variables such as cache hit rates, execution times, and relatedness of relevant data accessed during queries. We then compare these outcomes against theoretical baselines and random sequence generation. This comparative evaluation demonstrates that our query sequence simulation captures essential performance dynamics that existing benchmarks miss.

3.2. Simulation Methodology

Our methodology models three interdependent behaviors: task-oriented logical sessions, user interests for data locality, and refinement patterns for iterative query development.

3.2.1. Logical Query Sessions

To model logical sessions, we categorize queries into four primary topics: person-specific messages, forum information, person attributes, and message attributes. Because queries in decentralized environments often depend on preceding results, we map the output types of each template to valid subsequent templates. This creates a directed graph of query progression that governs the flow of a logical session.

Within a session, a template's *instantiation values* are typically derived from the previous query's results. To realistically simulate this "click-through" behavior, we execute centralized equivalents of the LDBC SNB queries using QLever [8]. We then randomly sample the returned result set to instantiate the subsequent query template. If a query yields no results, we fall back to our default, interest-based instantiation methodology.

As an example, consider the following data distributed across a decentralized environment:

Listing 1: (a) Alice's profile document (https://pod.example/alice/profile.ttl).

```
<https://pod.example/alice/profile.ttl#me> a :Person ;
    :knows <https://pod.example/bob/profile.ttl#me> ;
    :hasInbox <https://pod.example/alice/inbox> .
```

Listing 1: (b) Bob's profile document (https://pod.example/bob/profile.ttl).

```
<https://pod.example/bob/profile.ttl#me> a :Person ;
    :posts <https://pod.example/bob/posts/1> .
```

Listing 1: (c) Bob's post (https://pod.example/bob/posts/1).

```
<https://pod.example/bob/posts/1> a :Post ;
    :content "Hello Solid world!" .
```

Listing 1: (d) Alice's interests document (https://pod.example/alice/interests.ttl).

```
<https://pod.example/alice/profile.ttl#me> :interestedIn "Wildlife" .
```

Listing 1: (e) Wildlife forum document (https://pod.example/forum/wildlife.ttl).

```
<https://pod.example/forum/wildlife.ttl> a :Forum ;
    :hasPost <https://pod.example/ruben/posts/9> ;
    :hasComment <https://pod.example/bob/comment/1> ;
    :hasTopic "Wildlife" .
<https://pod.example/bob/comment/1> :responseTo <https://pod.example/ruben/posts/9> .
```

The generator randomly selects a user and simulates a realistic sequence of queries that reflects real-world user behavior. Given Given this dataset, and two query templates: one to fetch a person's posts and another to retrieve the content of a specific post, the generator can create a query sequence, which can contain multiple logical sessions. For user "alice", we can start the sequence (and first logical session) by executing the first query template (Q1) to discover Bob's posts, as shown in Listing 2.

Listing 2: Initial query (Q1) fetching Bob's posts.

```
SELECT ?post WHERE {
  <https://pod.example/bob/profile.ttl#me> :posts ?
      post .
}
```

The client-side query engine executes this query by traversing to Bob's profile document, returning the result binding: '?post = <https://pod.example/bob/posts/1>'.

To simulate a user clicking on this discovered post, the sequence generator instantiates the next query template dynamically. It extracts the value bound to "?post" from the results of Q1 and injects it as the subject of Q2, as shown in Listing 3.

Listing 3: Dynamic sequence query (Q2) instantiating a variable from Q1.

```
SELECT ?content WHERE {
  <https://pod.example/bob/posts/1> :content ?
      content .
}
```

This example demonstrates that the instantiation of Q2 strictly depends on the execution results of Q1. Because Q2 requires an instantiated post URI from Q1, these queries are directionally dependent; reversing their order leaves the generator with no method to instantiate the initial query parameters. Consequently, structured logical query sequences naturally emerge from these interdependencies, accurately reflecting realistic exploratory user behavior.

Finally, we simulate logical session switching. Drawing from web browsing behavior, we assign each sequence a session-switching probability sampled from a log-normal distribution with a mean of 15% [38, 51]. During sequence generation, a session switch triggers a random choice with equal probability: the engine either starts a new session (using interest-based instantiation) or resumes a previous one (deriving instantiation values from that session's last executed query). To prevent repetitive queries and maintain a uniform distribution of template instantiations, we explicitly prohibit backtracking within our model. Additionally, we maintain a balanced template distribution by dynamically scaling down each template's selection probability proportionally to its overall frequency.

Continuing our running example, a session switch occurs after executing Q2. Instead of continuing sequence generation from the content of Bob's post, the generator terminates the current path and selects a new template to instantiate: a query for posts within a forum.

As shown in Listing 4, the generator selects a wildlife forum as the initial instantiation value. This query requires an entirely separate data path to execute, breaking the dependency chain of the previous queries and representing a clear change in user intent. We are still generating the same *query sequence*, but the session switch introduces a new *logical session* within that sequence.

Listing 4: Independent query (Q3) initiating a session switch to explore a wildlife forum.

```
SELECT ?post WHERE {
  <https://pod.example/forum/wildlife.ttl> :hasPost
      ?post .
}
```

3.2.2. *User Interest Modeling*

Individuals interact more frequently with users sharing similar interests [40]. To simulate this behavior, we use the underlying LDBC dataset utilized by SolidBench [18], which inherently models realistic user preferences. The LDBC generator, Datagen, produces person profiles containing specific interests, educational backgrounds, and workplaces. It links users based on these demographic and interest overlaps, yielding a non-random network topology with a realistic degree distribution.

We use this interest-driven topology to generate template instantiation values for the first query of each logical session. Specifically, we construct a binary feature vector v_k for each user k:

$$v_k = (x_{k,1}, \ldots, x_{k,n}, y_{k,1}, \ldots, y_{k,m}), \tag{1}$$

where $x_{k,j} = 1$ if user k holds stated interest j, and $y_{k,m} = 1$ if user k completed study m.

For each sequence, we randomly select a base person k and compute the cosine similarity between v_k and the feature vectors of all other users. Let N be the total number of users. To maintain efficiency at large scale factors, we restrict our candidate pool to $C_{k,M}$, defined as the set of the top $M < N$ users ranked by similarity to k. Template instantiation values are then selected probabilistically using a softmax function with temperature T:

$$P(i \mid k) = \frac{e^{s_{k,i}/T}}{\sum_{u \in C_{k,M}} e^{s_{k,u}/T}}, \tag{2}$$

where $s_{k,i}$ is the similarity score between base person k and candidate $i \in C_{k,M}$. For templates requiring non-person values (e.g., posts or activities), selection probabilities are derived from the similarity scores of the content's creators.

Continuing our running example, consider an expanded decentralized environment containing a second forum dedicated to pottery, as shown in Listing 5 as well as all documents in Listing 1.

Listing 5: The additional pottery forum document (`https://pod.example/forum/pottery.ttl`).

```
<https://pod.example/forum/pottery.ttl> a :Forum ;
    :hasTopic "Pottery" .
```

To determine what instantiation value to use for the forum query for user Alice, we construct binary feature vectors over a small global feature space: (Wildlife, Pottery).

Alice's profile states an explicit interest in 'Wildlife', yielding vector $v_{\text{Alice}} = (1, 0)$. While the forums in the data have topic statements, thus the vectors for the forums are $v_{\text{Wildlife}} = (1, 0)$ and $v_{\text{Pottery}} = (0, 1)$.

This gives us a cosine similarity of $s_{\text{Alice,Wildlife}} = 1$ and $s_{\text{Alice,Pottery}} = 0$. Then using softmax with a temperature of $T = 0.5$, we compute the selection probabilities for the two forums:

$$P(\text{Wildlife} \mid \text{Alice}) = \frac{e^{1/0.5}}{e^{1/0.5} + e^{0/0.5}} \approx 0.88,$$
$$P(\text{Pottery} \mid \text{Alice}) = \frac{e^{0/0.5}}{e^{1/0.5} + e^{0/0.5}} \approx 0.12.$$

Consequently, when initiating or switching a session, the generator is more likely to select the 'Wildlife' forum for Alice, reflecting her interests.

3.2.3. Refinement Patterns

The benchmark simulates query refinement by applying composable transformations to an instantiated query template. Following our literature review, we focus on transformations (and their reciprocal counterparts) that add or modify BGPs, `OPTIONAL` clauses, `UNION` blocks, and query instantiation values.

To ensure these transformations meaningfully alter query execution rather than superficially appending triples, we apply a choke point-based design methodology [4]. The generator sequentially applies refinements to a base SolidSessionBench template. By mapping empirical exploratory usage patterns [5] to concrete planning (P) and traversal (T) choke points, we translate real-world user behaviors into rigorous execution bottlenecks for query engines.

The planning choke points, derived from empirical usage patterns [5] and relevant LDBC choke points [3], are defined as follows:

P.1 Involves adding or removing a `FILTER`, requiring prior knowledge to adjust query plans and estimate selectivity.

P.2 Addresses adding or removing `UNION` or `OPTIONAL` clauses to alter filter push-down capabilities and complicate query planning.

P.3 Covers adding or removing selective triple patterns, forcing the engine to identify changes and re-optimize the plan.

P.4 Focuses on adding or removing non-selective triple patterns.

Beyond planning, these refinements introduce traversal choke points that expand upon those established in SolidBench [57, 56]:

T.1 Expands or contracts the reachable sub-web through predicate additions.

T.2 Shifts the sub-web by substituting the traversal starting point.

T.3 Changes the number of hops required to obtain relevant data, corresponding to SolidBench choke points L.1–L.6 [56].

T.4 Alters the usability of structural indexes, such as type indexes, corresponding to SolidBench choke point L.8 [56].

These choke points directly map to the exploratory query behaviors identified in Section 2.2:

- **Result refinement** corresponds to P.1 and P.3, as it narrows results via `FILTER` constraints and selective triple patterns.
- **Result generalization** corresponds to P.3 and P.4, expanding existing BGPs with selective or non-selective triple patterns.
- **Result extension** maps to P.2, P.3, and P.4 by appending triple patterns to BGPs or introducing `OPTIONAL` and `UNION` blocks that alter filter push-down capabilities.
- **Shifting query intent** corresponds to T.1 and T.2, substituting instantiation values to initiate traversal from different seed documents, thereby shifting or expanding the reachable sub-web.
- **Query refinement** maps to P.2, modifying the structural complexity of the query by adding or removing `OPTIONAL` and `UNION` blocks.

Finally, the generator supports variable instantiation within refinements, allowing users to bind pattern variables to specific values randomly selected from configuration candidates. Users can define custom refinement patterns via JSON files to augment the default configurations.

An example of a slimmed-down default configuration is provided in Listing 6. This entry specifies an `OPTIONAL` refinement pattern that appends a triple pattern to retrieve the messages liked by a person.

The `location` parameter is set to 0, indicating that the triple pattern is added to the first existing `OPTIONAL` block, or to a new block if none is present. The default benchmark suite also includes a reciprocal configuration where `operation` is set to "removal", which instead deletes the triple pattern from the first `OPTIONAL` block.

Listing 6: Example configuration for an `OPTIONAL` refinement pattern.

```
{
  "type": "OPTIONAL",
  "id": 0,
  "operation": "addition",
  "description": "Add triple for likes (P.1, T.2)",
  "location": 0,
  "target": [
    {
      "subject": {
        "value": "person",
        "termType": "variable"
      },
      "predicate": {
        "value": ".../vocabulary/likes",
        "termType": "namedNode"
      },
```

Table 1
LogT-normal distribution parameters (μ, σ) control the queries per sequence, queries per logical session, and the probability of switching logical sessions. Refinement parameters define the likelihood (`patternProbability`) and bounds (`min/maxRefinementLength`) of generating a refinement sequence for a given query. Instantiation relies on a softmax `temperature` and a `maxLogits` cutoff for similarity-based entity selection.

Category	Parameter	Value
Distributions (μ, σ)	`Sequence Length`	3.0, 0.2
	`Session Length`	1.7, 0.5
	`Transition Probability`	-2.0, 0.25
Refinements	`patternProbability`	0.1
	`min/maxRefinementLength`	2, [3–5]
Instantiation	`temperature`	0.1
	`maxLogits`	5–10

```
      "object": {
        "value": "likedMessage",
        "termType": "variable"
      }
    }
  ]
}
```

All defaults, along with complete documentation, are available on GitHub.[1]

3.3. Hyperparameters

The sequence generation process introduces several new hyperparameters, which are detailed in Table 1 together with their default values. To guarantee reproducibility despite extensive random sampling, a single seeded pseudo-random number generator (PRNG) is used throughout the entire pipeline.

4. Caching in Link Traversal-based Query Processing

Unlike traditional query engines that evaluate queries against a centralized, pre-built index, LTQP evaluates queries directly over the live Web [56, 29] using the "follow-your-nose" principle.

Starting from a seed URI (often extracted from the query), the engine fetches data via a *link queue* (governed by *reachability criteria* [30]) and deposits the retrieved triples into an *active local store*. This local store is an in-memory dataset over which the query executes, typically utilizing dictionary encoding [55].

Previous work [27] established the fundamental concept of caching within this paradigm, demonstrating that a local repository of recently accessed Web documents mitigates the high network latency inherent to LTQP.

For our experiments, we use the Comunica LTQP engine [55, 57]. As a JavaScript-based engine, it relies on the event loop to asynchronously interleave HTTP requests during traversal and local query execution.

We use caching-based algorithms to validate SolidSessionBench query sequences and compare them against alternative benchmarking methods in the literature. Implementing these strategies establishes strong baselines for evaluating caching in traversal engines. This section introduces four cache implementations, each storing dereferenced triples differently. We also evaluate two cardinality estimation approaches that use cached triples to improve query planning and execution. Because these estimates require an indexed cache, we omit them for the unindexed baseline.

4.1. Cache Implementations

Unindexed LRU Cache This baseline caches raw triple arrays alongside source metadata. On a cache hit, the engine retrieves and indexes these triples into the active local store.

Indexed LRU Cache This approach pre-indexes triples and stores them in an in-memory LRU cache. On a hit, the engine extracts only the triples matching the query patterns. Combined with dictionary encoding, this pre-filtering reduces local store memory and indexing overhead, potentially offsetting the higher initial ingestion costs.

Memory-based Quad Store Cache This approach uses a quad store to unify cache indexes, saving memory and improving search efficiency. The quad graph stores the source of the triples, allowing cache hits to retrieve data using an '?subject ?predicate ?object' query where the graph term matches the cached document's URL.

Disk-based Quad Store Cache This approach stores the quad store on disk, providing the same advantages while supporting datasets larger than main memory. The quad store also supports count queries, however these count queries are not exact and instead provide count estimations.

4.2. Cardinality Estimation Approaches

Link traversal lacks prior knowledge regarding data distribution and availability, complicating query planning. To address this, we hypothesize that cached data provides sufficiently accurate cardinalities for query planning, and we evaluate two estimation approaches based on this cache. We integrate these estimates into the default Comunica cost-based optimizer [55] to generate more accurate query plans, replacing the zero-knowledge heuristics [28] traditionally used in link traversal.

Reachability-agnostic Cache-based Cardinality Estimation This approach estimates cardinalities across the entire cache. For each cache entry, it performs a query to count the number of matching triples and sums these counts to produce a final estimate.

Reachability-aware Cache-based Cardinality Estimation Here, we store the outgoing links of each cached document. For each query, we traverse the cache to find all

[1] https://github.com/SolidBench/SolidBench.js/tree/master/templates/refinements

documents reachable under the active reachability criterion, to produce more accurate estimates by ignoring irrelevant data. If the seed document is missing from the cache, we fall back to the Reachability-agnostic Cache-based Cardinality Estimation.

4.3. Experimental Configurations

For our experiments we evaluate three cache capacities: small (350k triples, ∼10% of the dataset), medium (700k, ∼20%), and large (1.4M, ∼40%). While the disk-based cache has a single size which stores 1.4M triples on disk.

Our implementations adhere to the RFC-9111 specification [22], meaning the cache correctly interprets standard HTTP headers (e.g., Cache-Control) just like a web browser would. However, because the benchmark dataset is static, cache entries require no revalidation and never become stale. Consequently, observed hit rates are likely higher than in a live environment.

Ultimately, comparing these strategies establishes 10 baselines: three algorithms per indexed cache (LRU, In-memory Quad Store and Disk-based Quad Store) and one algorithm for the unindexed cache, for caching performance in realistic user-centric environments, upon which future research into advanced caching approaches can build.

5. Evaluation

{**TODO**: fix floats}

We evaluate four client-side caching strategies to uncover their performance dynamics under realistic user behavior. To achieve this, we utilize SolidSessionBench to generate workloads that simulate empirical access patterns and logical query sessions. While we focus on caching due to its extensive application in client-side environments, the generated workloads support the evaluation of other personalized optimization techniques.

We structure the evaluation in four parts. First, we establish a performance baseline for the caching algorithms on the generated workloads. Next, we isolate logical sessions to quantify how session switching affects data locality and cache hit rates. We then analyze how query refinement patterns influence cache performance, examining both refinement depth and specific execution chokepoints. Finally, we contrast our outcomes with traditional evaluation methods, such as microbenchmarks and random sequences, to demonstrate that traditional workloads yield inaccurate caching performance estimates.

5.1. Experiment Setup

We evaluate our benchmark using the default scale of the SolidBench dataset (scale 0.1, 158,233 RDF files, 1,531 vaults, 3,556,159 triples). Using the default hyperparameters in Table 1, we generate 8 sequences totaling 192 queries. These queries are from the interactive workload, are split into discover and short queries, with multiple templates belonging to either of these groups. Experiments run on an Ubuntu 20.04 machine (Intel Core i5-9400) with 16 GB RAM allocated to the engine and a 180-second query timeout. We repeat each sequence 10 times to minimize variance and inject a uniform 70–90 ms latency to simulate a realistic network environment. The cache is scoped for a sequence, meaning no cache-state is shared between different executions or repetitions of sequences.

5.2. Evaluation on Full Query Workload

First, we present the overall performance characteristics of the tested caching algorithms on the default configuration of our introduced benchmark.

Unindexed LRU Cache In Figure 2, we find that the unindexed LRU cache reliably improves performance across all cache sizes, with larger caches yielding better results. For total execution time of queries, the large cache is significantly better than the smaller cache sizes, while for throughput the difference is less pronounced.

Indexed LRU Cache As shown in Figure 2, pre-indexing triples in the LRU cache accelerates successful query execution but introduces higher initial overhead. When high cache eviction rates outweigh the efficiency gains of indexing, this ingestion cost slows down execution or causes timeouts. The figure illustrates this overhead, as the no-cache baseline ultimately solves a higher percentage of queries than the standard indexed cache as the line reaches a higher y-value.

However, integrating cardinality estimation overcomes this limitation by enabling the query engine to generate more accurate query plans, increasing the percentage of solved queries and lowering execution times. Comparing the two estimation approaches, reachability-aware estimation outperforms reachability-agnostic estimation by ignoring irrelevant data. This effect is most pronounced in larger caches, where reachability-agnostic estimation yields only minor performance benefits, while reachability-aware estimation shows substantial improvements. This aligns with expectations: larger caches accumulate more irrelevant, out-of-context data, which the reachability-aware estimator successfully filters out.

Memory-based Quad store Cache As shown in Figure 3, the memory-based quad store cache exhibits high performance variance depending on the estimation strategy. Without estimation (*Indexed Store*), performance scales normally with capacity: the large cache achieves the best performance, while the medium and small caches sit lower but still reliably outperform the baseline. Similar to the LRU-based indexed cache, the baseline solves slightly more queries, indicating that the performance overhead can cause query failures. However, introducing online cardinality estimation (*Indexed Store (Estimate)*) completely inverts the performance scaling observed without estimation, with the small and medium caches outperforming the large cache.

The scaling nature of the performance penalty of cache size in estimation indicates that executing count queries scales unfavorably as the cached quad store size increases. Compared to the LRU-cache, which executes count triple pattern queries over small sources, the quad store-based

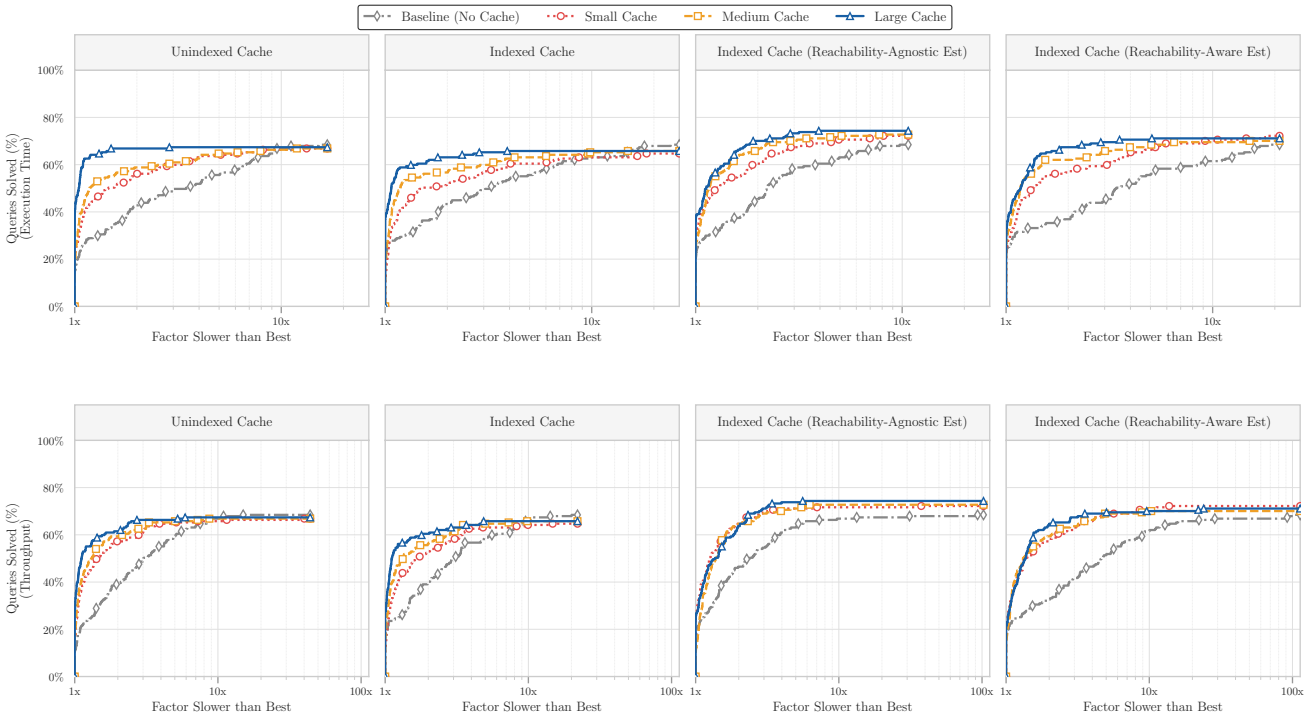

Figure 2: Performance profiles of the LRU-based unindexed and indexed triples cache. The y-axis shows the percentage of queries executing within a factor (x-axis) of the fastest algorithm. Curves closer to the top-left indicate better overall performance. The y-intercept ($x = 1$) shows how often an algorithm is the outright fastest, while the rightward trajectory demonstrates robustness (e.g., $x = 2, y = 80\%$ means 80% of queries execute within twice the fastest time). From the figure, we observe that all caching-based approaches outperform the no-cache baseline.

approach uses triple pattern queries over the entire quad store, but with the graph term filled in to represent the current document. For reachability-aware estimation the performance hit is not as big, especially in the total execution time case.

Disk-based Quad store Cache In Figure 3, we see that the disk-based quad store cache with size equal to large in-memory cache performs better than the baseline, however it under-performs or equals the small cache in most scenarios. Interestingly, disk-based estimation does not display the same performance hit compared to an in-memory cache of equal size, suggesting that the disk-based store implementation does not suffer the same scaling penalty. This indicates that the lack of large cache performance in the quad store case is a result of implementation inefficiency rather than algorithmic limitations. Generally, in memory constrained environments using disk-based cache will provide a performance benefit. However, it is much less pronounced than in-memory caching.

Comparison Between Caching Algorithms Comparing the four caching strategies reveals distinct operational trade-offs based on capacity and estimation requirements. The unindexed LRU cache provides the most consistent baseline improvement, scaling predictably with capacity to reduce total execution time. However, integrating cardinality estimation solves more queries while maintaining these speedup advantages.

Between the indexed approaches, the LRU cache scales better at larger capacities, whereas the in-memory quad store excels with smaller caches. Finally, the disk-based cache improves upon the baseline but is predictably outperformed by in-memory alternatives.

These findings highlight the potential of a tiered architecture: combining a small in-memory quad store with a large disk-based cache. Implementing this effectively requires a scan-resistant design that minimizes evictions and data movement between tiers.

5.3. Logical Session & Interest Modeling Cache Performance

To evaluate the impact of logical sessions on caching performance, we analyze the cache hit rate deviation (Δ) from the template baseline for queries initiating a new session, switching to an existing session, or remaining within the current session. We calculate this deviation on a per-template basis to control for inherent differences in query complexity.

Table 2 shows that new sessions severely degrade hit rates and existing session switches cause moderate degradation, whereas intra-session queries improve performance. These negative effects are most pronounced in small and

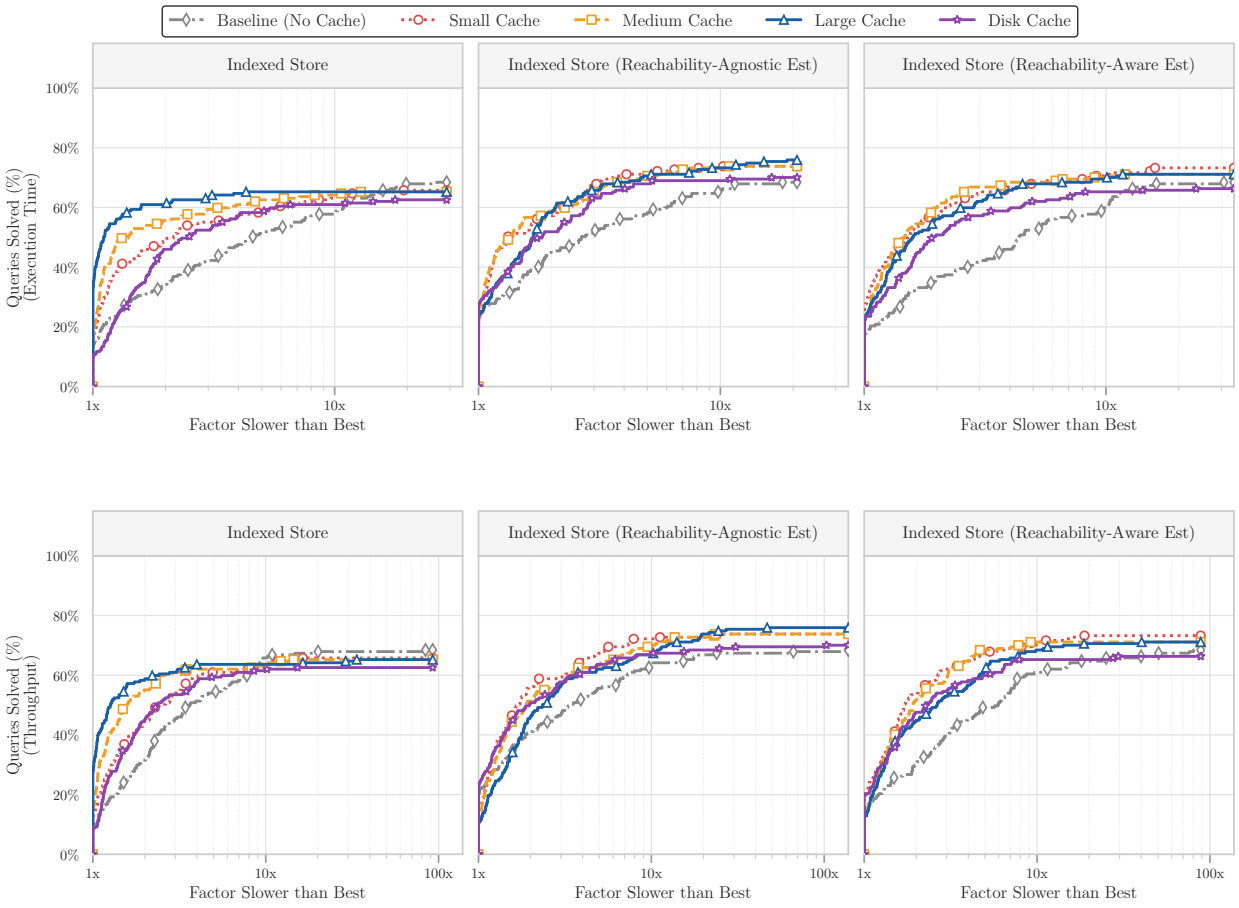

Figure 3: Performance profiles of the in-memory and disk-based quad store caches families of algorithms. The y-axis shows the percentage of queries executing within a factor (x-axis) of the fastest algorithm.

medium caches. Large caches retain performance better, though primarily when switching to an existing session.

Consequently, optimal client-side cache sizing depends heavily on session-switching behavior. These findings validate our sequence-based simulation design; ignoring session dynamics obscures critical performance shifts. They also highlight the need for eviction policies that retain cross-session entries.

Future work should investigate identifying and protecting a frequently accessed core of cache entries to mitigate session-switching penalties. Alternatively, query engines could identify logical sessions dynamically and maintain a separate cache for each. This ties in to the findings supporting the development of a hierarchical caching architecture, which requires strong cache resistance and supports caching previously identified logical sessions on-disk.

5.4. Refinement Patterns

We evaluate how our refinement patterns impact benchmark characteristics and challenge caching algorithms by analyzing the effect of individual chokepoints on query sequences and refinement depth on cache performance. Grounding these patterns in empirical data [5] already ensures they accurately reflect real-world user behavior. Consequently, this evaluation strategy primarily evaluates how these emperically found user behaviors affect caching performance and data locality.

5.4.1. Effect of chokepoints on query sequences

We isolate the structural impact of individual query chokepoints through a controlled experiment comprising 80 single-query sequences. Each sequence begins with a base query and sequentially applies a chain of two to five refinement patterns from the default set. We execute each sequence 10 times across three cache capacities (Small, Medium, Large). To analyze the chokepoints in the simplest configuration, we restrict this experiment to the LRU-based indexed cache. We measure cache hit rate, unique data sources accessed, and total execution time.

To quantify the performance impact of each chokepoint, we map the refinement patterns to their underlying chokepoints and evaluate them using a linear mixed-effects regression model. The model predicts the performance delta (ΔY) between the current and previous query step. We categorize chokepoint operations into additions (suffix ‘a’) and removals (suffix ‘r’). A removal operation occurs only when the sequence reverts a previously applied pattern. We represent the chokepoints applied in a given refinement step

Table 2
Impact of logical session status on algorithm performance. Values represent absolute and percentage cache hit rate deviations from the global template average across new, existing, and within-session queries. Absolute values represent the mean deviation across queries, and percentages are calculated using the ratio of averages (mean absolute deviation divided by the mean baseline). Negative values indicate worse cache performance for these query types.

Cache Algorithm	New Session		Existing Session		Within Session	
	Abs.	%	Abs.	%	Abs.	%
lru unindexed-l	-0.092	-15.557	-0.059	-9.824	0.035	6.013
lru unindexed-m	-0.109	-24.034	-0.045	-9.720	0.029	6.555
lru unindexed-s	-0.091	-24.120	-0.066	-17.139	0.039	10.460
lru indexed-l	-0.080	-13.715	-0.044	-7.404	0.027	4.586
lru indexed-m	-0.080	-17.202	-0.057	-11.917	0.034	7.101
lru indexed-s	-0.085	-21.823	-0.066	-16.458	0.039	9.869
in-memory store-l	-0.193	-29.819	-0.017	-2.644	0.019	3.060
in-memory store-m	-0.137	-28.394	-0.020	-4.103	0.018	3.810
in-memory store-s	-0.108	-29.691	-0.056	-15.100	0.035	9.697
disk store-l	-0.070	-12.966	-0.044	-8.028	0.028	5.052

as a binary vector $\mathbf{T}_{i,t}$. We use these vectors as predictors, alongside the previous query's performance metrics ($Y_{i,t-1}$), to account for the system's baseline state. The regression model is:

$$\Delta Y_{i,t} = \gamma Y_{i,t-1} + \mathbf{T}_{i,t}^{\top}\boldsymbol{\beta} + u_i + \epsilon_{i,t} \tag{3}$$

where $\Delta Y_{i,t}$ is the performance delta for sequence i at refinement step t, $Y_{i,t-1}$ is the previous step's independent variable value, $\mathbf{T}_{i,t}$ is a binary indicator vector of applied chokepoints, $\boldsymbol{\beta}$ is the vector of fixed effects estimating the isolated impact of each pattern, u_i is a sequence-specific random intercept absorbing unobserved heterogeneity, and $\epsilon_{i,t}$ is the residual error.

Figure 4 plots the estimated effect of each chokepoint on performance. Full regression tables are available in Appendix A. We observe three primary behaviors:

First, modifying specific patterns degrades performance in both directions. For example, adding or removing `T.3` (increasing required traversal hops) reduces the cache hit rate and increases execution time. This indicates an invalidation penalty: the chokepoint forces the cache to evict useful data, preventing recovery when the chokepoint is removed. Consequently, caching in these environments requires scan-resistant algorithms.

Second, patterns such as `T.2` (new instantiation values) and `P.4` (non-selective triple patterns) exhibit reversible effects. Adding these chokepoints reduces performance, but removing them improves the hit rate and accelerates execution. In these instances, the cache successfully maintains temporal locality across the refinement, isolating the performance degradation to the chokepoint itself.

Third, the large cache counterintuitively yields a smaller positive effect on the hit rate for chokepoints like `P.3.r` (selective triple patterns), `P.2.r` (unions or optionals), and `P.1.a` (filters) compared to smaller caches. Because the hit rate has an upper bound of 1.0, the large cache has less margin for improvement. The average hit rates across all queries support this ceiling effect: 0.51, 0.61, and 0.66 for the Small, Medium, and Large caches, respectively. The significant negative coefficient of the lagged hit rate in our model further confirms that a higher previous hit rate limits the expected delta.

Finally, across all patterns, chokepoints that increase the number of accessed data sources consistently reduce the cache hit rate.

Implications for Refinement Pattern Design This analysis exposes structural weaknesses in standard caching algorithms. We demonstrate that structural mutations like `T.3` trigger severe invalidation penalties. Conversely, the reversibility of patterns like `T.2` proves that caches can maintain temporal locality despite changes to the traversal seed.

Mapping refinement patterns to specific chokepoints allows researchers to isolate these structural impacts and directly evaluate caching algorithms. Because our sequence generation is dynamically configurable, it acts as a diagnostic tool. Developers can extend our default configurations to isolate targeted chokepoints, enabling custom stress tests for distinct caching architectures.

5.4.2. Effect of Refinement Depth on Performance

To understand the impact of exploratory query refinement on cache performance, we evaluate how our caching strategies behave as refinement depth increases. We define depth as the number of consecutive refinement patterns applied to a base query. By measuring average execution times and cache hit rates at varying depths, we reveal how caches manage highly correlated, sequential workloads.

Figure 5 shows that cache hit rates predictably rise as refinement depth increases, proving that caching algorithms successfully exploit the structural similarities inherent to

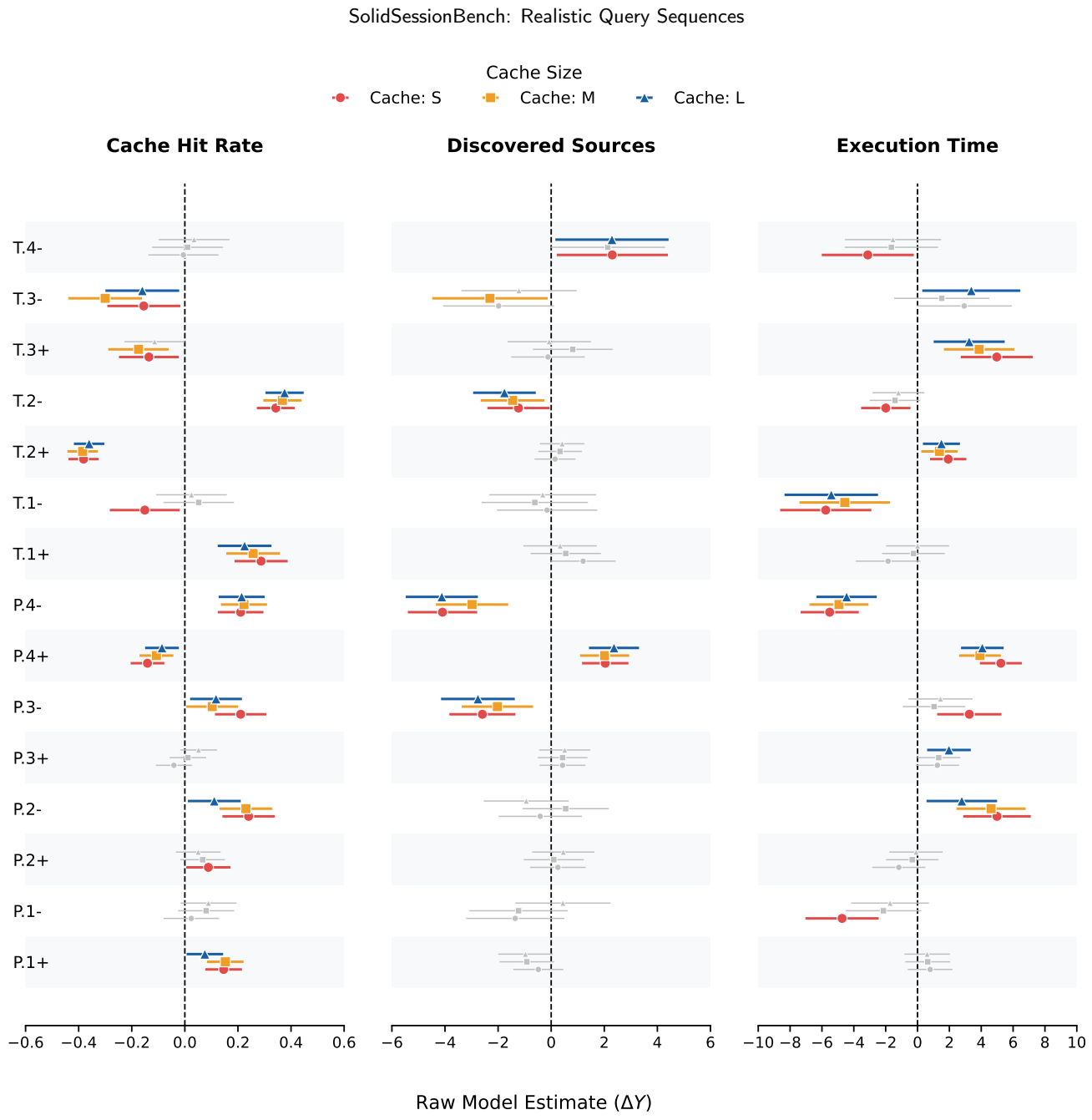

Figure 4: Marginal fixed effects of refinement pattern additions (+) and removals (-) on query performance across Small (S), Medium (M), and Large (L) cache capacities. The plot reports the unstandardized absolute expected change (ΔY) for Cache Hit Rate, Discovered Sources, and Execution Time. For both discovered sources and execution time, the plot uses signed log-counts and signed log-milliseconds, respectively. Error bars represent 95% confidence intervals and statistically insignificant estimates ($p \geq 0.05$) are rendered in gray and any chokepoints without any significant coefficients are omitted. Undoing the traversal chokepoint changing the instantiation value of the query significantly improves cache hit rate (+.35) and reduces the number of unique data sources accessed. However, its effect on execution time is on the edge of significance.

exploratory refinement. However, at a refinement depth of 5, hit rates drop sharply alongside a spike in evictions. This indicates that deeper exploratory refinements sufficiently differ from the starting query so that either new subwebs must be fetched or previously relevant data has been evicted. Furthermore, while the reachability-agnostic and no-estimation algorithms report high hit rates, the reachability-aware approach reports significantly lower rates. This is an artifact of the estimation strategy: the aware estimator performs an offline traversal of the cache to build its plan, logging internal cache misses when it probes links at the edge of the cached graph that have not yet been dereferenced. Consequently, its reported hit rate is artificially deflated by these planning-phase checks, suggesting the actual HTTP-request driven hit rate is likely comparable to the other approaches.

Figure 6 demonstrates that an increased hit rate does not reliably translate to faster execution or higher throughput. While caching generally improves these metrics, speedup variance remains high across the sequence. Performance drops considerably at depth 3 and spikes at depth 5, which coincides with lower hit rates. Notably, disk-based caches and small or medium caches using reachability-aware cardinality estimation achieve massive performance gains at depth 5, yielding up to a 100× speedup over the baseline. Interestingly, this effect disappears for large, non-disk caches.

We attribute this anomaly to three factors. First, variance increases at larger depths because fewer refinement sequences reach depth 5. Second, the cost model is unpredictable. Smaller caches hold less data, which degrades the accuracy of their cardinality estimates. Counterintuitively, these poor estimates occasionally cause the optimizer to select a vastly superior join plan. Third, the queries exhibiting extreme throughput improvements contain a LIMIT clause (e.g., interactive-discover-8). This allows traversal to terminate early once the efficient plan produces the required results.

However, these factors do not fully explain the performance spike for the disk-based quad store. Given the potential for massive speedups, future work must isolate the root cause of this behavior and determine if it can be systematically applied to other areas of query execution.

5.5. Impact of Workload Realism on Caching Performance

The final step in our evaluation contrasts the performance outcomes obtained under realistic user behavior with those derived from conventional caching evaluation methods, specifically synthetic microbenchmarks and random query sequences. Prior caching research heavily relies on these traditional workloads. However, by comparing these approaches to our sequence-based evaluation, we demonstrate that conventional methods different and overly pessimistic conclusions regarding cache performance in decentralized environments.

5.6. Comparison to Alternative Evaluation Methods

The final step in our evaluation compares the performance outcomes obtained from SolidSessionBench against alternative evaluation methods commonly found in literature, specifically synthetic microbenchmarks and random query sequences. By demonstrating that these conventional methods yield divergent and often misleading conclusions regarding cache performance, we validate the necessity of our emperical data-based query sequence simulation.

Microbenchmarks-Theoretical Hit Rate Analysis Cache literature extensively uses microbenchmarks of various forms to analyze caching performance [41, 27]. To compare the conclusions of such a benchmark to the conclusions of the benchmark introduced in this paper we investigate theoretical caching speedup without management costs, by executing each query template twice with simulated cache miss probabilities. We excluded the estimation-based approaches, as using the full cache for estimation would skew the results for miss rates above zero.

Figure 7 shows representative performance profiles for the workload. Most templates (8 out of 12) show reliable improvement that scales with the hit rate for both algorithms. However, certain long-running queries (e.g., interactive-discover-6, 7, and short-3) show no speedup, and some, such as interactive-short-3, even experience slowdowns. These templates traverse many Solid pods but do not require all traversed data. We hypothesize that rapidly serving documents from an in-memory cache overwhelms the system with data, slowing result production. Conversely, HTTP network latency gives the JavaScript event loop time to process the join execution pipeline, yielding initial results faster since only the first few requests are needed. The disk-based cache is an outlier on discover-8, showing major slowdowns for time-to-first-result. Raw data analysis reveals that the query's LIMIT clause causes inaccurate measurements. Because the disk-based cache requires longer ingestion times, fewer documents are dereferenced before reaching the limit. Subsequent queries therefore experience cache misses before the limit is met, making the measured speedups unrepresentative of a true 100% cache hit rate.

Generally, an indexed cache (store-based or LRU) outperforms an unindexed cache. However, this approach incurs higher cache management costs, which theoretical hit rates do not account for. Although slightly slower than the in-memory quad store, the disk-based quad store cache remains competitive with the other approaches. This demonstrates that disk-based caching is effective in memory-constrained environments. The conclusions from this evaluation differ from the full SolidSessionBench benchmark results because they hide the significant overhead penalty of managing a disk-based cache. While the full benchmark shows the disk-based cache performs significantly worse than all in-memory caches, this evaluation method suggests their performance is similar. The full figures are found in Appendix B.

Random Query Sequences Another common benchmarking approach is to randomly order queries from an existing benchmark into a sequence [42, 11, 27, 41, 23]. Figures 9 and 8 show execution times and throughput for sequences created by randomly grouping default SolidBench.js queries. Without cardinality estimation, only the large cache improves over the baseline, while medium, small, and disk-based caches perform similarly or worse. With cardinality estimation (both reachability-agnostic and -aware), no cache reliably outperforms the baseline, though the reachability-aware approach yields minor improvements when using the large cache. The exception is the reachability-aware disk-based cache, as this cache suffers less from the estimation performance penalty seen in the in-memory store. These results differ significantly from our benchmark results, leading to different conclusions about the utility of caching. Because random query sequences ignore the literature on established

Figure 5: Cache hit ratio and eviction percentage across refinement depths (0 to 5). The grid separates configurations by cardinality estimation strategy (rows) and cache capacity (columns). Line plots denote the cache hit ratio, while the shaded background bars represent the percentage of cache capacity evicted during the sequence.

application user behavior, they produce overly pessimistic estimates of caching performance in real-world scenarios.

Scope and Limitations We evaluate SolidSessionBench against microbenchmarks and random sequences, but explicitly exclude hand-crafted scenarios and simple distributions. Hand-crafted methods lack scalability, while simple distributions fail to capture complex user habits. By automating logical sessions and user interests, our benchmark functions as a scalable superset of both approaches, making direct comparison redundant. By contrast, the compared methods do not incorporate these sequence attributes, and our results show this omission risks drawing inaccurate conclusions.

6. Conclusion

In this paper, we presented a comprehensive evaluation of client-side caching strategies for Link Traversal Query Processing (LTQP) in decentralized environments. By utilizing SolidSessionBench to simulate realistic query sequences, we uncovered critical client-side performance characteristics that traditional evaluation methods fail to find. Our evaluation yielded three primary insights. First, logical session switching severely degrades cache hit rates, proving that optimal cache sizing and eviction policies must account for session dynamics. Second, deep exploratory query refinement introduces significant cache invalidation challenges while exposing unexploited optimization opportunities. For instance, inaccurate cost models and cardinality

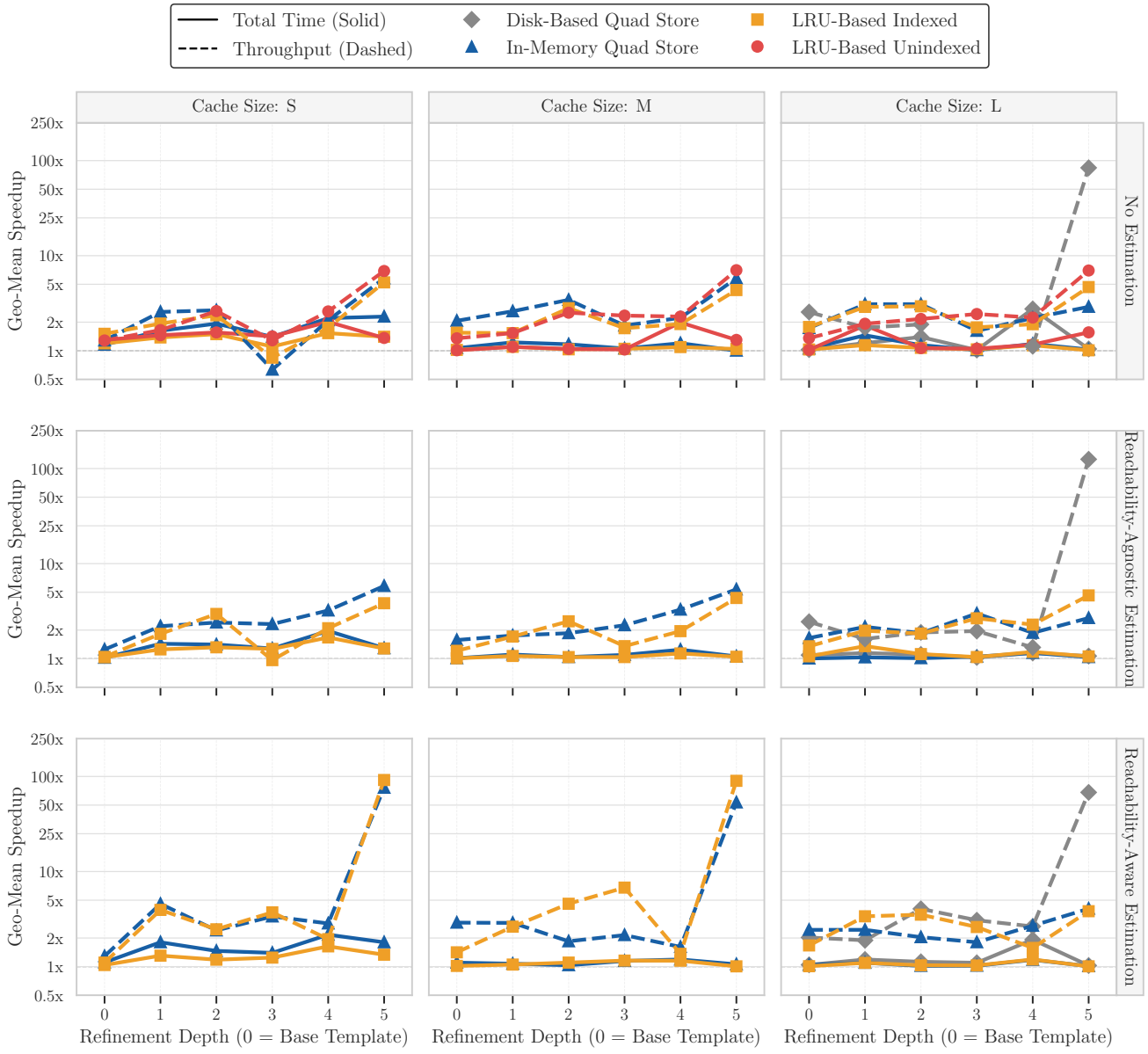

Figure 6: Geometric mean speedup of total execution time (solid lines) and query throughput (dashed lines) relative to the no-cache baseline. Results are faceted by cardinality estimation strategy (rows) and cache capacity (columns). The Reachability-Aware Estimation row and Large cache column utilize a logarithmic y-axis to accommodate significant performance gains at higher refinement depths.

estimates in disk-based and smaller caches occasionally trigger highly efficient plan switches, yielding speedups exceeding 100×. Third, cache-based triple pattern cardinality estimation effectively optimizes link traversal queries by providing the prior knowledge traditionally absent in LTQP.

These findings demonstrate that traditional benchmarking methods, such as microbenchmarks and random query sequences, produce misleading and overly pessimistic estimates of real-world caching performance. To realize the full potential of caching in LTQP, client-side query engines must move beyond static, session-agnostic architectures. Future research should prioritize session-aware caching mechanisms that dynamically identify and protect cross-session core entries. Furthermore, developing hierarchical architectures, such as combining small in-memory quad stores with large disk-based caches, can mitigate the performance penalties of session switching. Finally, future work should explore alternative cache-based cardinality estimation methods, particularly for evaluating join cardinalities.

To ensure longevity and support future research into client-side optimization, SolidSessionBench is integrated into the core SolidBench ecosystem. The framework is modular, MIT-licensed, and fully documented, with all code, data, and experiment setups available at `https://github.com/RubenEschauzier/simulated-query-sequence-paper`.

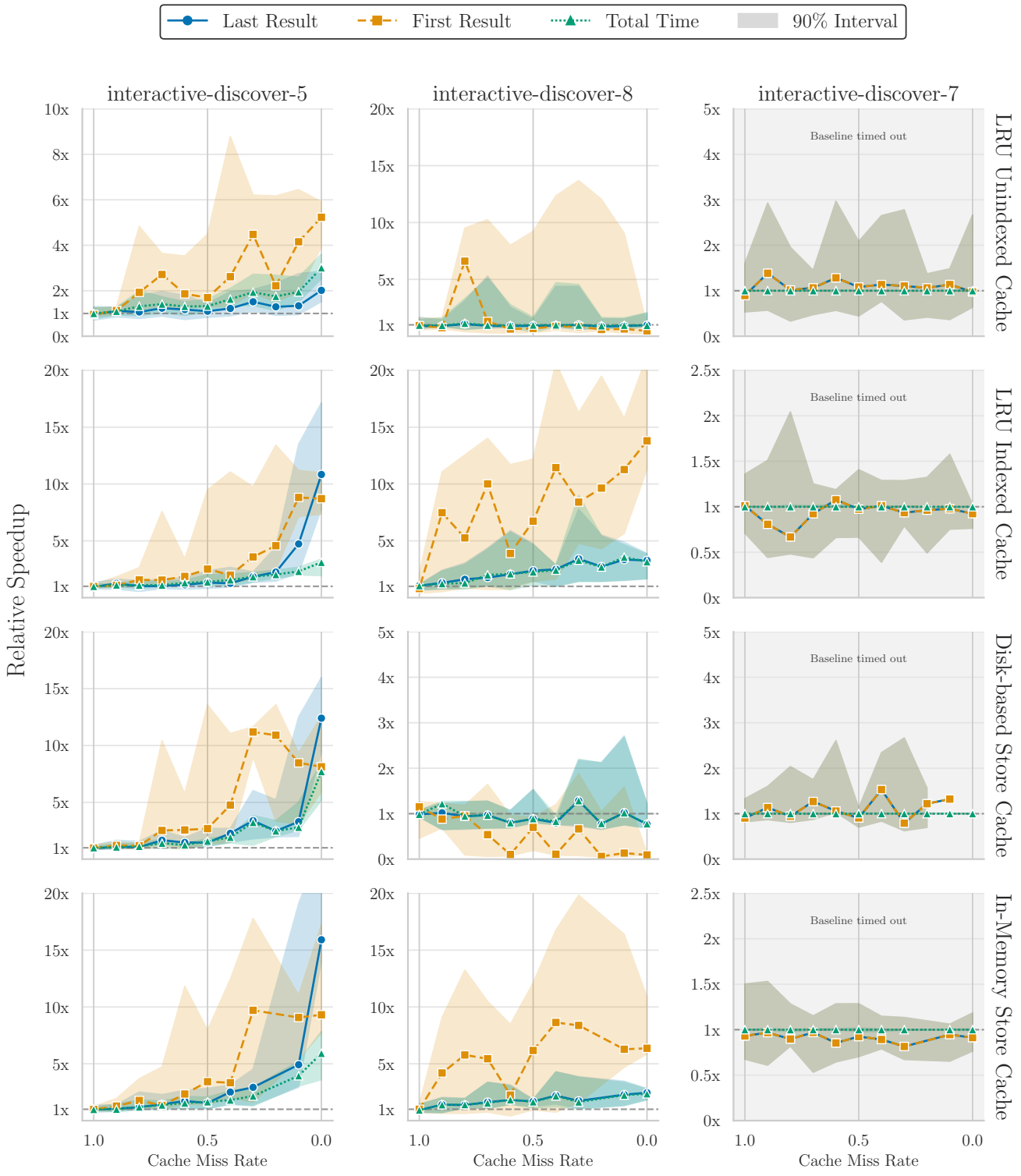

Figure 7: Performance of LRU-based unindexed, LRU-based indexed, In-memory store, and Disk-based store caches across representative query templates. The y-axis shows the mean speedup relative to a baseline with a 0% hit rate. Shaded regions denote the 90th percentile

CRediT authorship contribution statement

Ruben Eschauzier: TODO:. **Ruben Taelman:** TODO:.

References

[1] Aluç, G., Hartig, O., Özsu, M.T., Daudjee, K., 2014. Diversified stress testing of rdf data management systems, in: International Semantic Web Conference, Springer. pp. 197–212.

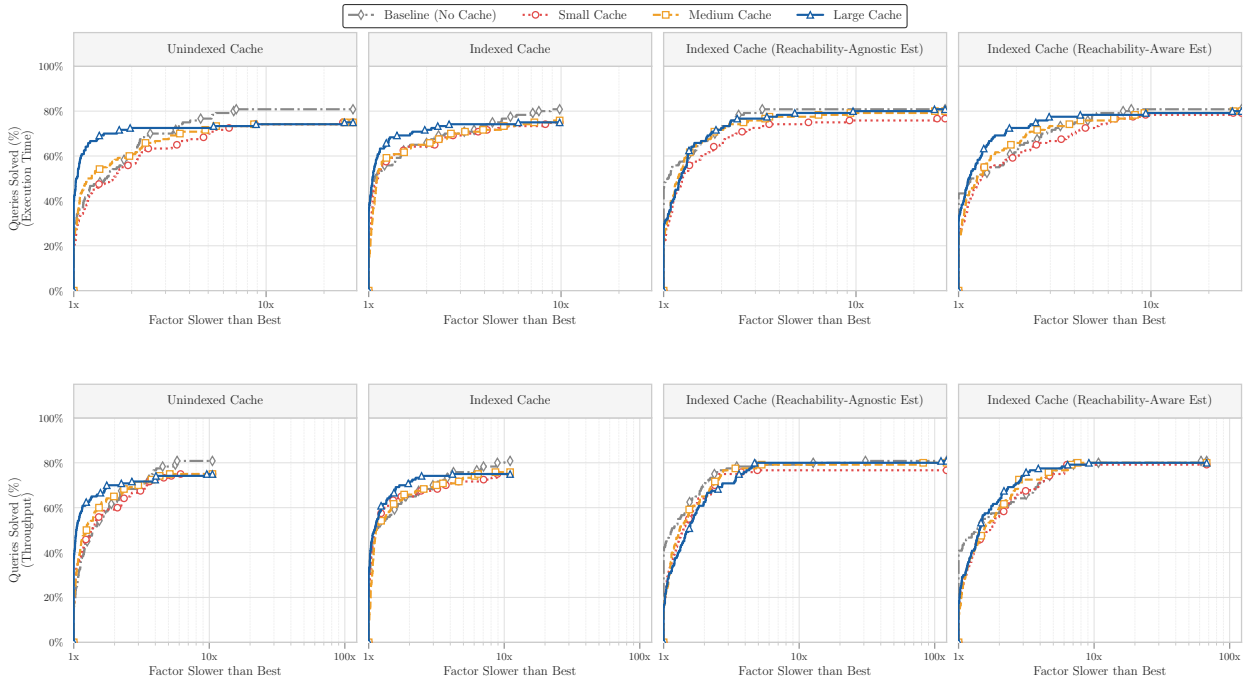

Figure 8: Performance profiles of the lru-based unindexed and indexed caches families of algorithms on random query sequences. The y-axis shows the percentage of queries executing within a factor (x-axis) of the fastest algorithm.

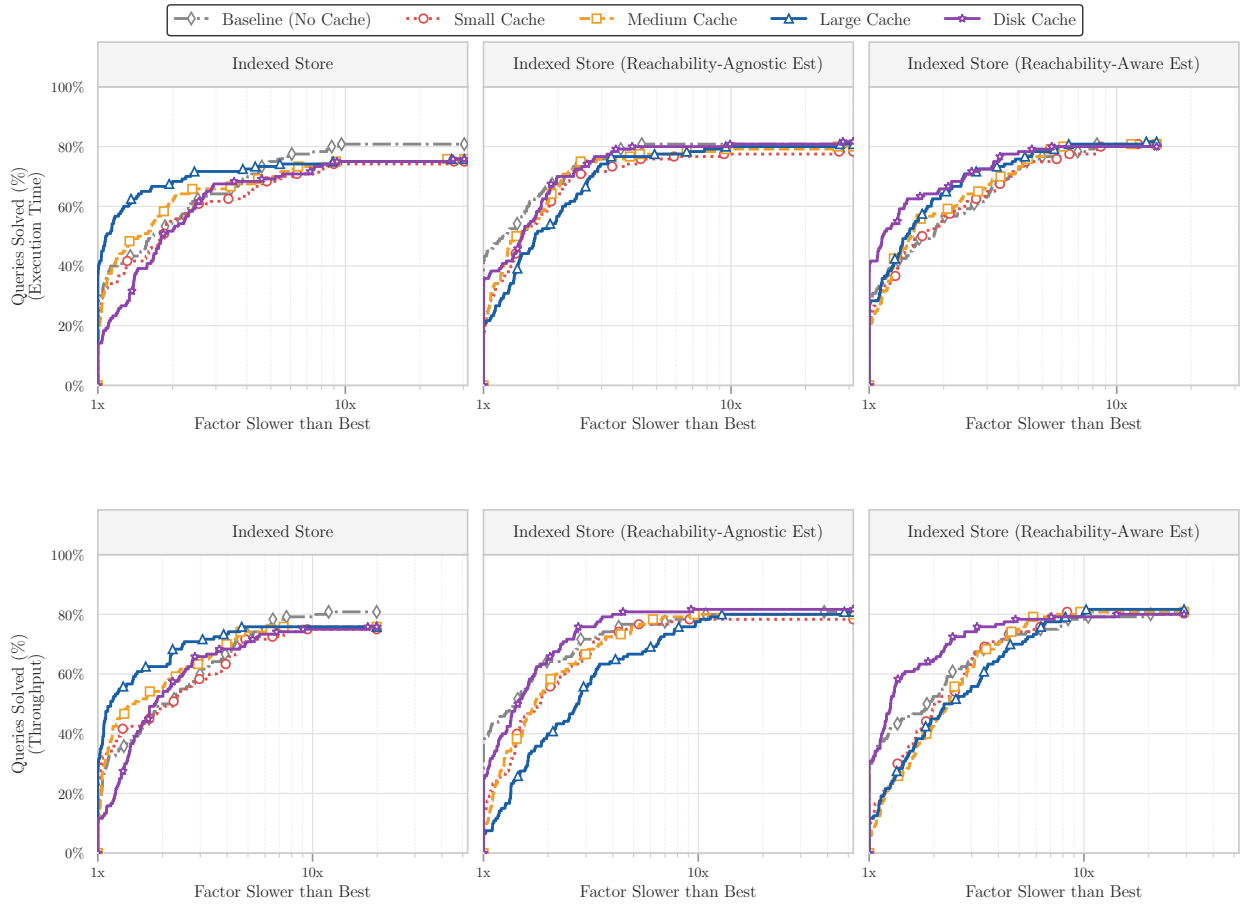

Figure 9: Performance profiles of the in-memory and disk-based quad store caches families of algorithms on random query sequences. The y-axis shows the percentage of queries executing within a factor (x-axis) of the fastest algorithm.

[2] Amblard, F., Bouadjio-Boulic, A., Gutiérrez, C.S., Gaudou, B., 2015. Which models are used in social simulation to generate social networks? a review of 17 years of publications in jasss, in: 2015 Winter Simulation Conference (WSC), IEEE. pp. 4021–4032.

[3] Angles, R., Antal, J.B., Averbuch, A., Birler, A., Boncz, P., Búr, M., Erling, O., Gubichev, A., Haprian, V., Kaufmann, M., et al., 2020. The ldbc social network benchmark.

[4] Angles, R., Boncz, P., Larriba-Pey, J., Fundulaki, I., Neumann, T., Erling, O., Neubauer, P., Martinez-Bazan, N., Kotsev, V., Toma, I., 2014. The linked data benchmark council: a graph and rdf industry benchmarking effort, ACM New York, NY, USA. pp. 27–31.

[5] Arenas, M., Franconi, E., Hammerer, J., Hartig, O., Hose, K., Koesten, L., Konstantinidis, G., Libkin, L., Martens, W., Sasaki, Y., et al., 2025. Exploring exploratory querying. Proceedings of the VLDB Endowment 18, 5731–5739.

[6] Arnold, B.C., 2014. Pareto distribution. Wiley StatsRef: Statistics Reference Online , 1–10.

[7] Backstrom, L., Huttenlocher, D., Kleinberg, J., Lan, X., 2006. Group formation in large social networks: membership, growth, and evolution, in: Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining, pp. 44–54.

[8] Bast, H., Buchhold, B., 2017. Qlever: A query engine for efficient sparql+ text search, in: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, pp. 647–656.

[9] Berendt, B., Hollink, L., Luczak-Rösch, M., Möller, K., Vallet, D., 2013. Usewod2013 3rd international workshop on usage analysis and the web of data. 10th ESWC-Semantics and Big Data, Montpellier, France .

[10] Berners-Lee, T., 2017. Charlie: An AI that works for you. W3C Design Issues. URL: https://www.w3.org/DesignIssues/Charlie.html. updated 2023.

[11] Bizer, C., Schultz, A., 2011. The berlin sparql benchmark, in: Semantic Services, Interoperability and Web Applications: Emerging Concepts. IGI Global Scientific Publishing, pp. 81–103.

[12] Boncz, P., Neumann, T., Erling, O., 2013. Tpc-h analyzed: Hidden messages and lessons learned from an influential benchmark, in: Technology Conference on Performance Evaluation and Benchmarking, Springer. pp. 61–76.

[13] Bonifati, A., Martens, W., Timm, T., 2020. An analytical study of large sparql query logs. The VLDB Journal 29, 655–679.

[14] Bronson, N., Amsden, Z., Cabrera, G., Chakka, P., Dimov, P., Ding, H., Ferris, J., Giardullo, A., Kulkarni, S., Li, H., et al., 2013. {TAO}:{Facebook's} distributed data store for the social graph, in: 2013 USENIX annual technical conference (USENIX ATC 13), pp. 49–60.

[15] Chatzopoulos, S., Vergoulis, T., Skoutas, D., Dalamagas, T., Tryfonopoulos, C., Karras, P., 2022. Atrapos: Evaluating metapath query workloads in real time. CoRR .

[16] Clark, A., 2007. Understanding community: A review of networks, ties and contacts .

[17] Dang, M.H., Aimonier-Davat, J., Molli, P., Hartig, O., Skaf-Molli, H., Le Crom, Y., 2023. Fedshop: A benchmark for testing the scalability of sparql federation engines, in: International Semantic Web Conference, Springer. pp. 285–301.

[18] Erling, O., Averbuch, A., Larriba-Pey, J., Chafi, H., Gubichev, A., Prat, A., Pham, M.D., Boncz, P., 2015. The ldbc social network benchmark: Interactive workload, in: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, pp. 619–630.

[19] Eschauzier, R., Taelman, R., 2026. SolidSessionBench: Realistic query sequences for user-oriented decentralized environments. Conditionally accepted to ISWC 2026. Camera-ready preprint available at https://rubeneschauzier.github.io/simulated-query-sequence-paper/build/main.pdf.

[20] Eschauzier, R., Taelman, R., Verborgh, R., 2025. Revisiting link prioritization for efficient traversal in structured decentralized environments, in: International Semantic Web Conference, Springer. pp. 575–593.

[21] Esteves, B., Pandit, H.J., Rodríguez-Doncel, V., 2021. Odrl profile for expressing consent through granular access control policies in solid, in: 2021 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), IEEE. pp. 298–306.

[22] Fielding, R., Nottingham, M., Reschke, J., 2022. Rfc 9111: Http caching.

[23] Folz, P., Skaf-Molli, H., Molli, P., 2016. Cyclades: a decentralized cache for triple pattern fragments, in: European semantic web conference, Springer. pp. 455–469.

[24] Hamill, L., Gilbert, G., 2009. Social circles: A simple structure for agent-based social network models. Journal of Artificial Societies and Social Simulation 12.

[25] Hanski, J., Taelman, R., Verborgh, R., 2024. Observations on bloom filters for traversal-based query execution over solid pods, in: European Semantic Web Conference, Springer. pp. 228–233.

[26] Hanski, J., Van Braeckel, S., Taelman, R., Verborgh, R., 2025. Link traversal over decentralised environments using restart-based query planning, in: International Conference on Web Engineering.

[27] Hartig, O., 2011a. How caching improves efficiency and result completeness for querying linked data., in: LDOW.

[28] Hartig, O., 2011b. Zero-knowledge query planning for an iterator implementation of link traversal based query execution, in: Extended Semantic Web Conference, Springer. pp. 154–169.

[29] Hartig, O., 2012. Sparql for a web of linked data: Semantics and computability, in: Extended Semantic Web Conference, Springer. pp. 8–23.

[30] Hartig, O., Bizer, C., Freytag, J.C., 2009. Executing sparql queries over the web of linked data, in: International semantic web conference, Springer. pp. 293–309.

[31] Hartig, O., Özsu, M.T., 2016. Walking without a map: Ranking-based traversal for querying linked data, in: International Semantic Web Conference, Springer. pp. 305–324.

[32] Heling, L., Acosta, M., 2022. Federated sparql query processing over heterogeneous linked data fragments, in: Proceedings of the ACM Web Conference 2022, pp. 1047–1057.

[33] Kalaitzakis, A., Papadakis, H., Fragopoulou, P., 2012. Evolution of user activity and community formation in an online social network, in: Proceedings of the 2012 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining, IEEE. pp. 1315–1320.

[34] Karki, N., Pandey, M., Tiwari, S., Mihindukulasooriya, N., Groppe, S., Dobriy, D., 2026. Agentic ai, context engineering and knowledge graphs: Current approaches, challenges and opportunities., in: EDBT/ICDT Workshops.

[35] Lorey, J., Naumann, F., 2013. Caching and prefetching strategies for sparql queries, in: Extended Semantic Web Conference, Springer. pp. 46–65.

[36] Malyshev, S., Krötzsch, M., González, L., Gonsior, J., Bielefeldt, A., 2018. Getting the most out of wikidata: Semantic technology usage in wikipedia's knowledge graph, in: The Semantic Web–ISWC 2018: 17th International Semantic Web Conference, Monterey, CA, USA, October 8–12, 2018, Proceedings, Part II 17, Springer. pp. 376–394.

[37] Martin, M., Unbehauen, J., Auer, S., 2010. Improving the performance of semantic web applications with sparql query caching, in: Extended Semantic Web Conference, Springer. pp. 304–318.

[38] Meiss, M., Duncan, J., Gonçalves, B., Ramasco, J.J., Menczer, F., 2009. What's in a session: tracking individual behavior on the web, in: Proceedings of the 20th ACM conference on Hypertext and hypermedia, ACM, Torino Italy. pp. 173–182. URL: https://dl.acm.org/doi/10.1145/1557914.1557946, doi:10.1145/1557914.1557946.

[39] Mislove, A., Marcon, M., Gummadi, K.P., Druschel, P., Bhattacharjee, B., 2007. Measurement and analysis of online social networks, in: Proceedings of the 7th ACM SIGCOMM conference on Internet measurement, pp. 29–42.

[40] Mitzlaff, F., Atzmueller, M., Benz, D., Hotho, A., Stumme, G., 2013. User-relatedness and community structure in social interaction networks. arXiv preprint arXiv:1309.3888 .

[41] Nicholson, H., Chrysogelos, P., Ailamaki, A., 2024. Hpcache: memory-efficient olap through proportional caching revisited. The VLDB Journal 33, 1775–1791.

[42] O'Neil, P., O'Neil, E., Chen, X., Revilak, S., 2009. The star schema benchmark and augmented fact table indexing, in: Technology Conference on Performance Evaluation and Benchmarking, Springer. pp. 237–252.

[43] Padia, A., Finin, T., Joshi, A., et al., 2015. Attribute-based fine grained access control for triple stores, in: 3rd Society, Privacy and the Semantic Web-Policy and Technology workshop, 14th International Semantic Web Conference.

[44] Papailiou, N., Tsoumakos, D., Karras, P., Koziris, N., 2015. Graph-aware, workload-adaptive sparql query caching, in: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, pp. 1777–1792.

[45] Prat-Pérez, A., Dominguez-Sal, D., 2014. How community-like is the structure of synthetically generated graphs?, in: Proceedings of Workshop on GRAph Data management Experiences and Systems, pp. 1–9.

[46] Raman, A., Joglekar, S., Cristofaro, E.D., Sastry, N., Tyson, G., 2019. Challenges in the decentralised web: The mastodon case, in: Proceedings of the internet measurement conference, pp. 217–229.

[47] Sadagopan, N., Li, J., 2008. Characterizing typical and atypical user sessions in clickstreams, in: Proceedings of the 17th international conference on World Wide Web, pp. 885–894.

[48] Saleem, M., Ali, M.I., Hogan, A., Mehmood, Q., Ngomo, A.C.N., 2015. Lsq: the linked sparql queries dataset, in: International semantic web conference, Springer. pp. 261–269.

[49] Sambra, A.V., Mansour, E., Hawke, S., Zereba, M., Greco, N., Ghanem, A., Zagidulin, D., Aboulnaga, A., Berners-Lee, T., 2016. Solid: a platform for decentralized social applications based on linked data. MIT CSAIL & Qatar Computing Research Institute, Tech. Rep. 2016.

[50] Schmidt, M., Görlitz, O., Haase, P., Ladwig, G., Schwarte, A., Tran, T., 2011. Fedbench: A benchmark suite for federated semantic data query processing, in: The Semantic Web–ISWC 2011: 10th International Semantic Web Conference, Bonn, Germany, October 23-27, 2011, Proceedings, Part I 10, Springer. pp. 585–600.

[51] Schneider, F., Feldmann, A., Krishnamurthy, B., Willinger, W., 2009. Understanding online social network usage from a network perspective, in: Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement, pp. 35–48.

[52] Schwarte, A., Haase, P., Hose, K., Schenkel, R., Schmidt, M., 2011. Fedx: Optimization techniques for federated query processing on linked data, in: The Semantic Web–ISWC 2011: 10th International Semantic Web Conference, Bonn, Germany, October 23-27, 2011, Proceedings, Part I 10, Springer. pp. 601–616.

[53] Stadler, C., Saleem, M., Mehmood, Q., Buil-Aranda, C., Dumontier, M., Hogan, A., Ngonga Ngomo, A.C., 2024. Lsq 2.0: A linked dataset of sparql query logs. Semantic Web 15, 167–189.

[54] Taelman, R., 2024. Towards applications on the decentralized web using hypermedia-driven query engines. ACM SIGWEB Newsletter URL: https://www.rubensworks.net/raw/publications/2024/towards_query_engines_decentralized_hypermedia.pdf.

[55] Taelman, R., Van Herwegen, J., Vander Sande, M., Verborgh, R., 2018. Comunica: a modular sparql query engine for the web, in: International Semantic Web Conference, Springer. pp. 239–255.

[56] Taelman, R., Verborgh, R., 2023a. Evaluation of link traversal query execution over decentralized environments with structural assumptions.

[57] Taelman, R., Verborgh, R., 2023b. Link traversal query processing over decentralized environments with structural assumptions, in: International Semantic Web Conference, Springer. pp. 3–22.

[58] Tam, B.E., Taelman, R., Colpaert, P., Verborgh, R., 2024. Opportunities for shape-based optimization of link traversal queries. arXiv preprint arXiv:2407.00998 .

[59] Umbrich, J., Hose, K., Karnstedt, M., Harth, A., Polleres, A., 2011. Comparing data summaries for processing live queries over linked data. World Wide Web 14, 495–544.

[60] Vögele, C., van Hoorn, A., Schulz, E., Hasselbring, W., Krcmar, H., 2018. Wessbas: extraction of probabilistic workload specifications for load testing and performance prediction—a model-driven approach for session-based application systems. Software & Systems Modeling 17, 443–477.

[61] Walter, S., Bast, H., 2025. Grasp: Generic reasoning and sparql generation across knowledge graphs, in: International Semantic Web Conference, Springer. pp. 271–289.

[62] Williams, G.T., Weaver, J., 2011. Enabling fine-grained http caching of sparql query results, in: International Semantic Web Conference, Springer. pp. 762–777.

[63] Zhang, W.E., Sheng, Q.Z., Qin, Y., Yao, L., Shemshadi, A., Taylor, K., 2016. Secf: improving sparql querying performance with proactive fetching and caching, in: Proceedings of the 31st Annual ACM Symposium on Applied Computing, pp. 362–367.

[64] Zhang, X., Wang, M., Zhao, B., Liu, R., Zhang, J., Yang, H., 2020. Characterizing robotic and organic query in sparql search sessions, in: Web and Big Data: 4th International Joint Conference, APWeb-WAIM 2020, Tianjin, China, September 18-20, 2020, Proceedings, Part I 4, Springer. pp. 270–285.

Table 3
Fixed effects and goodness-of-fit for cache hit rate. Standard errors in parentheses.

	S	M	L
$Y_{i,t-1}$	-1.007*** (0.025)	-0.982*** (0.024)	-0.949*** (0.024)
P-1+	0.146*** (0.035)	0.152*** (0.035)	0.075* (0.035)
P-1-	0.024 (0.053)	0.080 (0.054)	0.089 (0.054)
P-2+	0.089* (0.043)	0.067 (0.043)	0.050 (0.043)
P-2-	0.240*** (0.050)	0.230*** (0.051)	0.111* (0.051)
P-3+	-0.041 (0.035)	0.012 (0.035)	0.052 (0.035)
P-3-	0.210*** (0.049)	0.103* (0.050)	0.117* (0.050)
P-4+	-0.141*** (0.032)	-0.107*** (0.032)	-0.086** (0.032)
P-4-	0.210*** (0.044)	0.223*** (0.044)	0.214*** (0.044)
T-1+	0.287*** (0.051)	0.257*** (0.052)	0.225*** (0.052)
T-1-	-0.151* (0.067)	0.052 (0.068)	0.025 (0.068)
T-2+	-0.382*** (0.029)	-0.385*** (0.029)	-0.361*** (0.029)
T-2-	0.343*** (0.037)	0.368*** (0.037)	0.376*** (0.037)
T-3+	-0.135* (0.057)	-0.174** (0.058)	-0.114 (0.058)
T-3-	-0.155* (0.070)	-0.300*** (0.071)	-0.160* (0.071)
T-4+	0.068 (0.057)	0.043 (0.058)	0.072 (0.057)
T-4-	-0.006 (0.068)	0.010 (0.068)	0.035 (0.068)
Log-Likelihood	-362.336	-379.669	-378.989
Marginal R^2	0.284	0.234	0.222
Conditional R^2	0.913	0.922	0.920
ICC	0.878	0.898	0.897

A. Full Regression Tables

This appendix provides the complete statistical results from the linear mixed-effects regression model used to quantify the performance impact of each query chokepoint.

Table 3 presents the fixed effects (β) of chokepoint additions (+) and removals (-) on the cache hit rate.

Table 4 lists the isolated structural impacts and fixed effects for Discovered Sources.

Table 5 outlines the fixed effects for the Execution Time metric.

Table 4
Fixed effects and goodness-of-fit for cache hit rate. Standard errors in parentheses.

	S	M	L
$Y_{i,t-1}$	-0.099 (0.058)	-0.170** (0.065)	-0.162* (0.072)
P-1+	-0.485 (0.481)	-0.916 (0.524)	-0.971 (0.524)
P-1-	-1.354 (0.943)	-1.228 (0.947)	0.442 (0.914)
P-2+	0.253 (0.534)	0.097 (0.577)	0.456 (0.598)
P-2-	-0.414 (0.801)	0.545 (0.829)	-0.938 (0.816)
P-3+	0.427 (0.441)	0.430 (0.477)	0.510 (0.492)
P-3-	-2.592*** (0.635)	-2.026** (0.685)	-2.759*** (0.709)
P-4+	2.036*** (0.446)	2.012*** (0.473)	2.366*** (0.480)
P-4-	-4.095*** (0.666)	-2.981*** (0.695)	-4.120*** (0.693)
T-1+	1.203 (0.623)	0.549 (0.676)	0.336 (0.706)
T-1-	-0.156 (0.964)	-0.617 (1.021)	-0.319 (1.026)
T-2+	0.144 (0.393)	0.335 (0.420)	0.415 (0.428)
T-2-	-1.231* (0.598)	-1.453* (0.613)	-1.761** (0.602)
T-3+	-0.119 (0.707)	0.813 (0.765)	-0.074 (0.802)
T-3-	-1.979 (1.066)	-2.306* (1.111)	-1.214 (1.107)
T-4+	-0.588 (0.687)	-1.032 (0.751)	-1.109 (0.784)
T-4-	2.302* (1.068)	2.134 (1.100)	2.289* (1.088)
Log-Likelihood	-4048.816	-4046.863	-4000.735
Marginal R^2	0.155	0.129	0.152
Conditional R^2	0.214	0.240	0.329
ICC	0.070	0.127	0.209

Table 5
Fixed effects and goodness-of-fit for execution time. Standard errors in parentheses.

	S	M	L
$Y_{i,t-1}$	-0.810*** (0.111)	-0.698*** (0.085)	-0.669*** (0.084)
P-1+	0.791 (0.716)	0.640 (0.717)	0.605 (0.730)
P-1-	-4.737*** (1.174)	-2.139 (1.214)	-1.723 (1.245)
P-2+	-1.174 (0.851)	-0.330 (0.842)	-0.087 (0.854)
P-2-	4.990*** (1.082)	4.619*** (1.106)	2.781* (1.130)
P-3+	1.245 (0.695)	1.331 (0.689)	1.973** (0.700)
P-3-	3.252** (1.030)	1.040 (1.002)	1.443 (1.026)
P-4+	5.233*** (0.671)	3.918*** (0.667)	4.069*** (0.682)
P-4-	-5.515*** (0.933)	-4.930*** (0.944)	-4.452*** (0.966)
T-1+	-1.850 (1.037)	-0.255 (1.007)	0.002 (1.011)
T-1-	-5.762*** (1.461)	-4.566** (1.450)	-5.415*** (1.497)
T-2+	1.926*** (0.582)	1.376* (0.581)	1.501* (0.593)
T-2-	-1.991* (0.791)	-1.411 (0.811)	-1.196 (0.830)
T-3+	4.974*** (1.152)	3.868*** (1.130)	3.243** (1.137)
T-3-	2.930 (1.523)	1.522 (1.528)	3.375* (1.571)
T-4+	1.104 (1.141)	0.561 (1.132)	0.352 (1.149)
T-4-	-3.122* (1.474)	-1.642 (1.495)	-1.542 (1.539)
Log-Likelihood	-4345.931	-4381.203	-4413.325
Marginal R^2	0.247	0.209	0.202
Conditional R^2	0.556	0.455	0.432
ICC	0.411	0.311	0.288

B. Full Synthetic Hit Rate Figures

This section provides the full results of the synthetic hit rate experiments summarized in Section 5. Figures 10, 11, 12, and 13 show the results for the LRU-based unindexed cache, LRU-based indexed cache, the in-memory store cache, and the disk-based store cache, respectively.

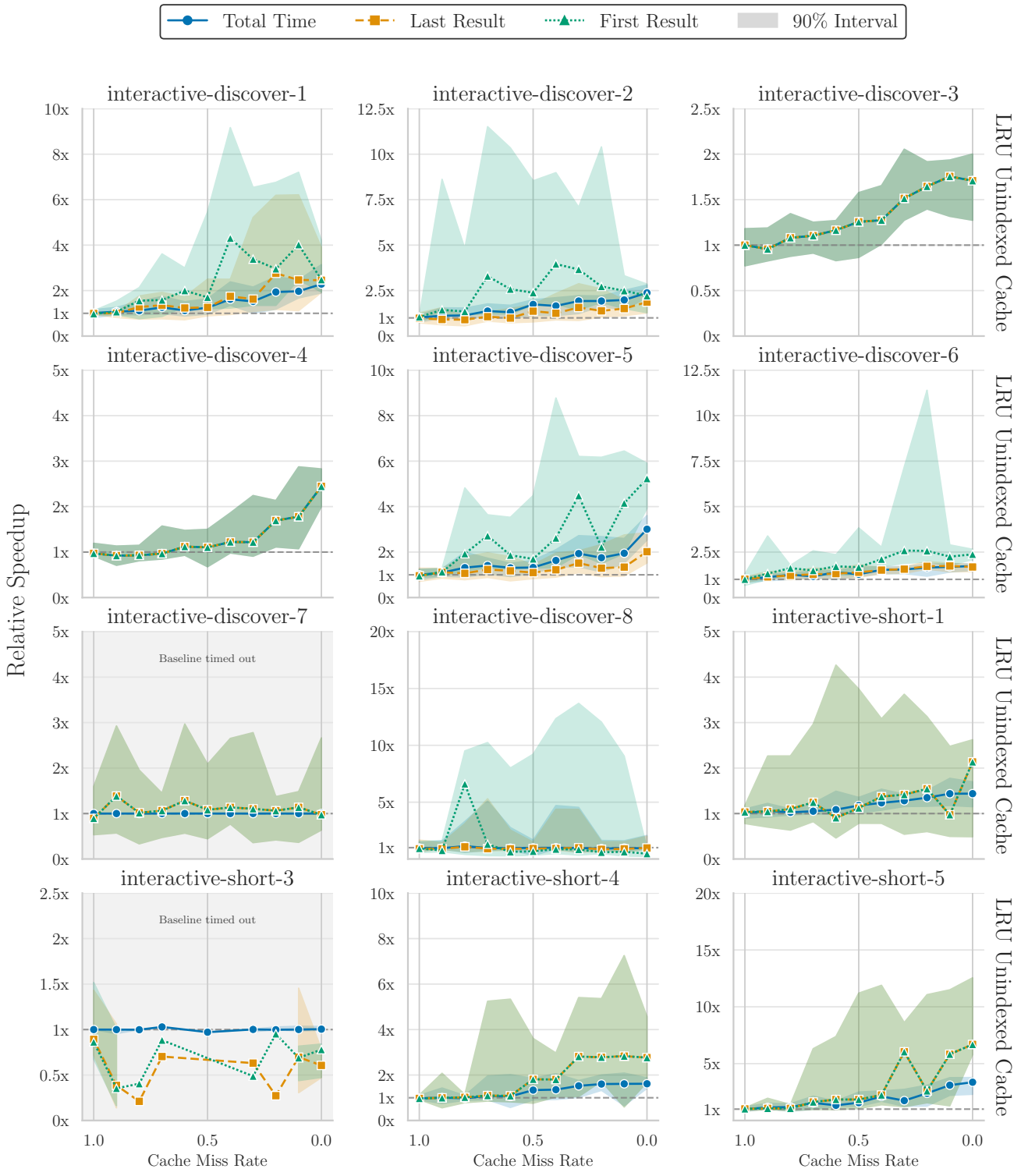

Figure 10: Performance of LRU-based unindexed cache across all query templates. The y-axis shows the mean speedup relative to a baseline with a 0% hit rate. Shaded regions denote the 90th percentile

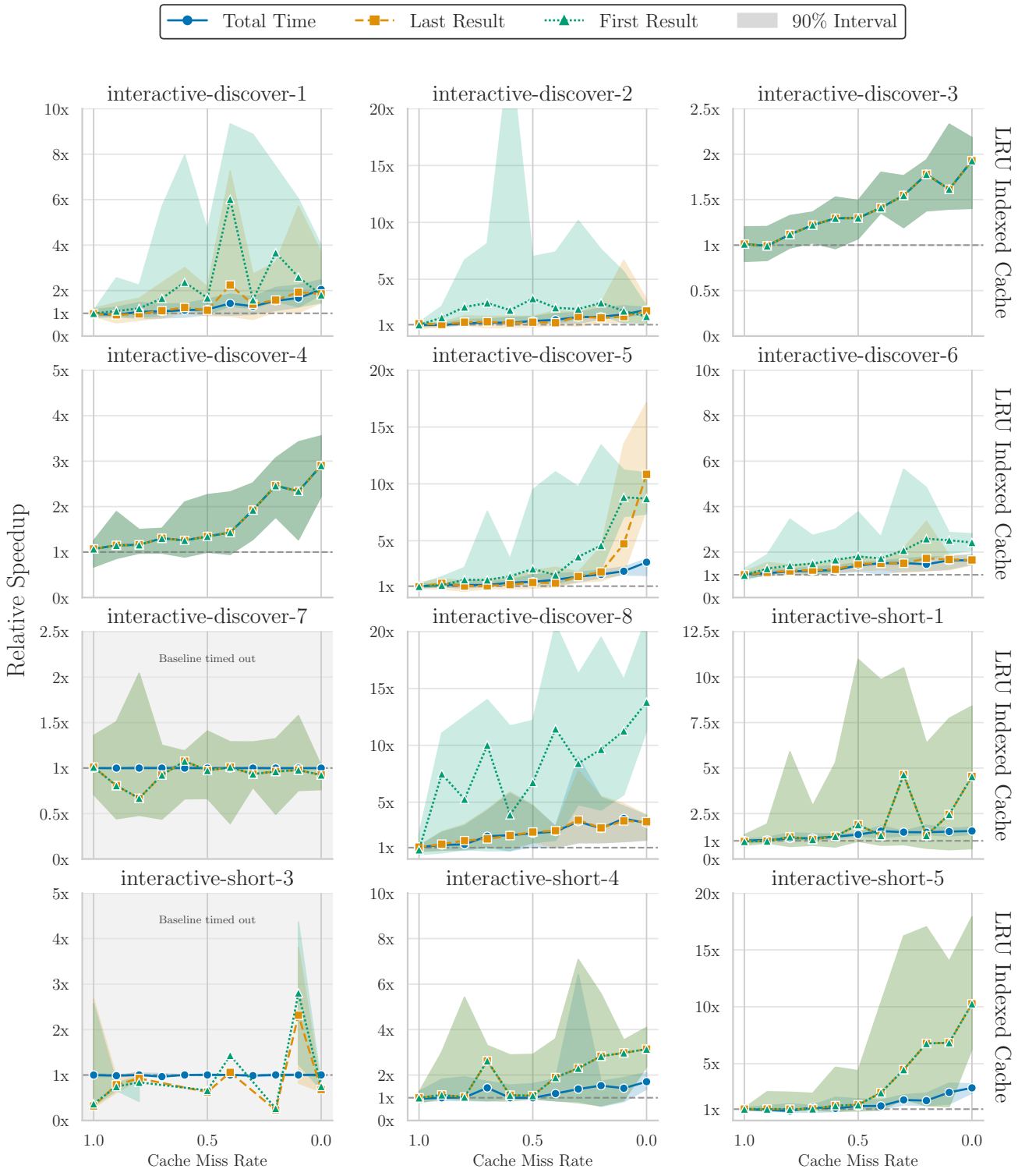

Figure 11: Performance of LRU-based indexed cache across all query templates. The y-axis shows the mean speedup relative to a baseline with a 0% hit rate. Shaded regions denote the 90th percentile

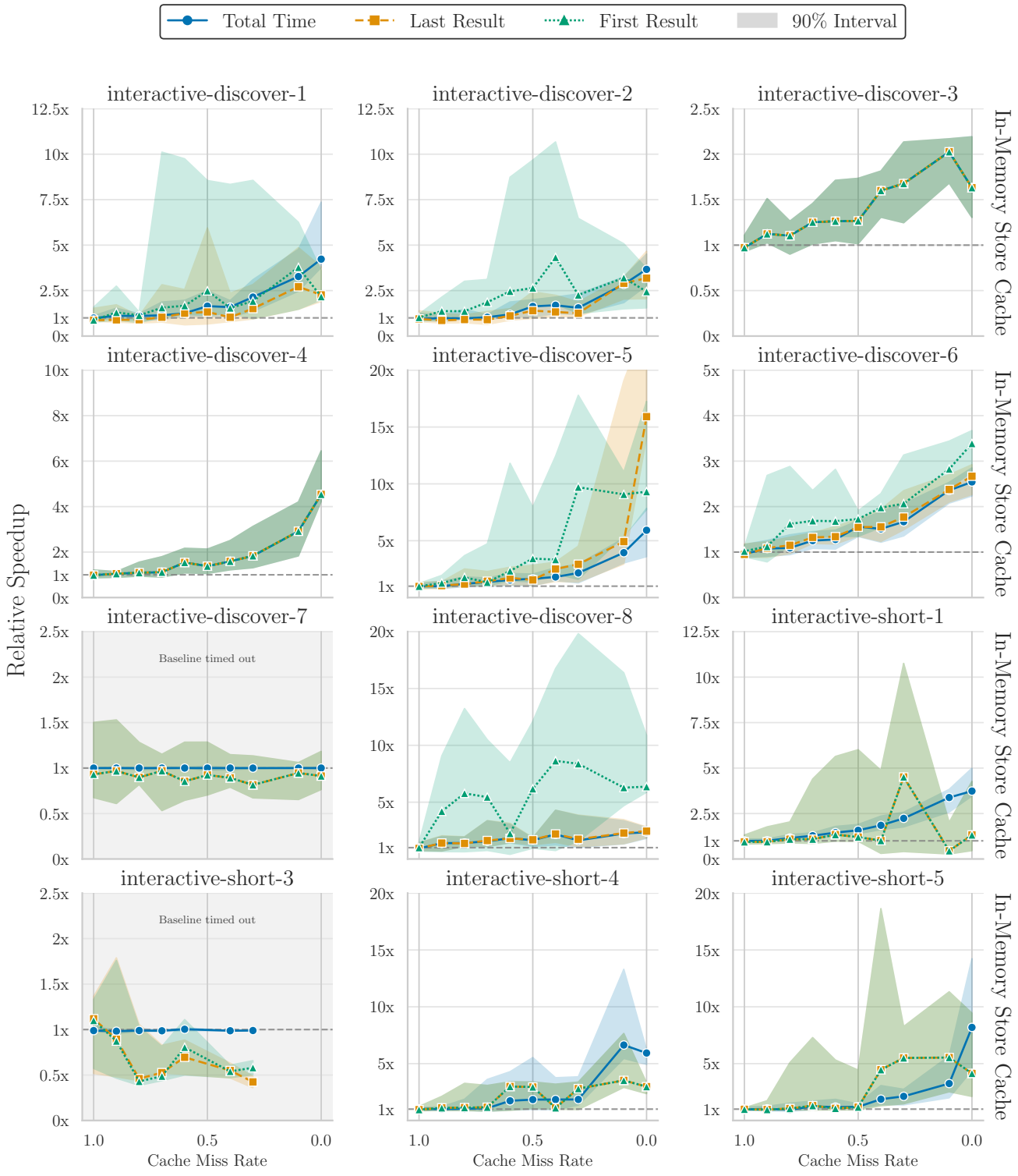

Figure 12: Performance of in-memory store cache across all query templates. The y-axis shows the mean speedup relative to a baseline with a 0% hit rate. Shaded regions denote the 90th percentile

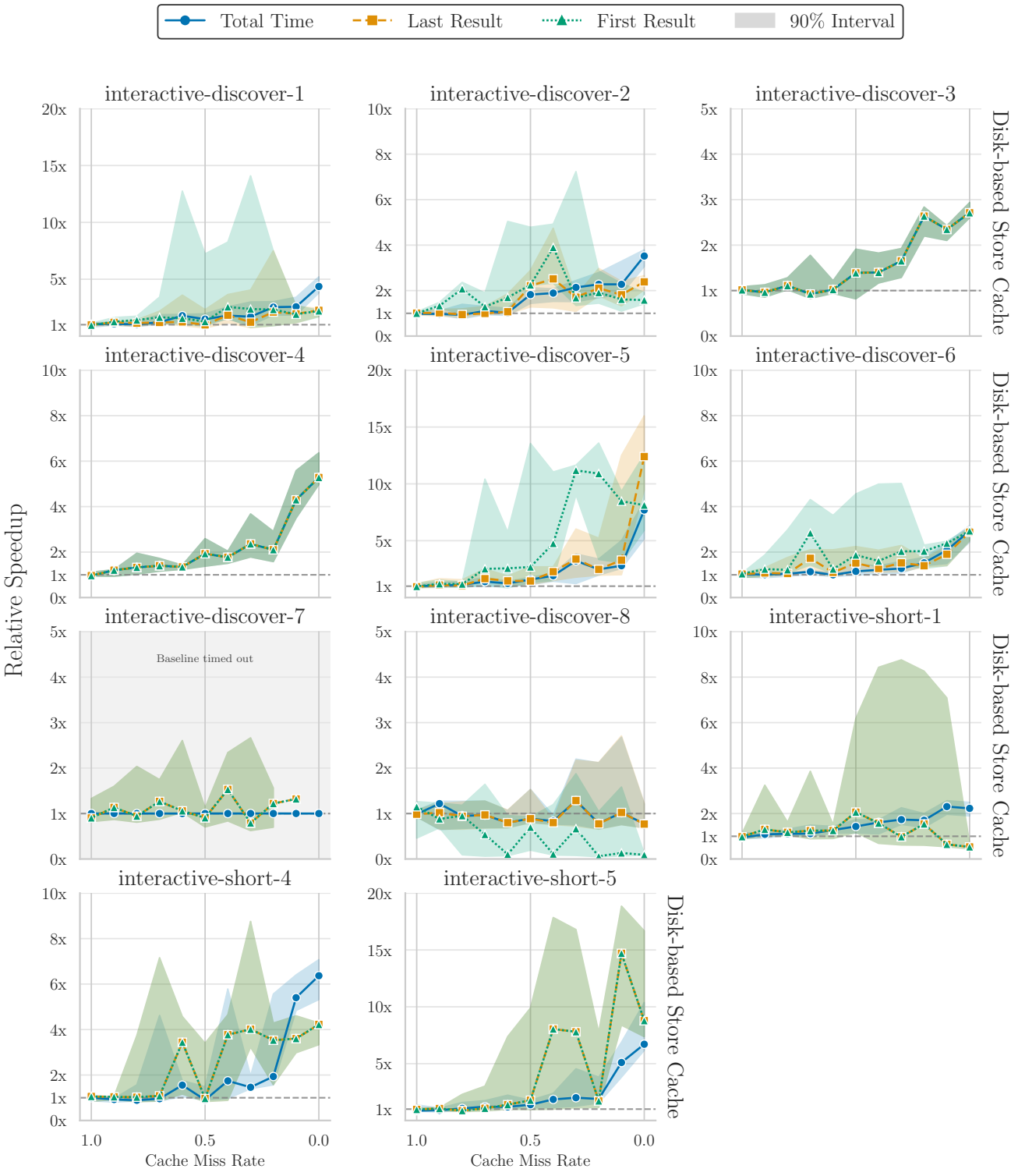

Figure 13: Performance of disk-based store cache across all query templates. The y-axis shows the mean speedup relative to a baseline with a 0% hit rate. Shaded regions denote the 90th percentile